Algoritmos em Computação Quântica

Felipe Junqueira Leite
Orientador: Prof. Dr. André Martin Timpanaro

Universidade Federal do ABC (UFABC)

Centro de Ciência, Matemática, Computação e Cognição (CMCC)

Setembro 2025 – Setembro 2026

Resumo

Este trabalho apresenta um estudo prático e teórico dos fundamentos da computação quântica e de seus principais algoritmos. O objetivo central é transpor a barreira entre o formalismo matemático do Espaço de Hilbert e a implementação de circuitos lógicos funcionais. Utilizando a linguagem Python e o *framework* Qiskit, foram desenvolvidos e simulados experimentos contemplando a criação dos quatro Estados de Bell, a verificação empírica da Desigualdade CHSH e o emaranhamento multipartido (GHZ). Adicionalmente, o trabalho detalha a arquitetura e a execução de algoritmos quânticos avançados, especificamente o Algoritmo de Grover para busca não estruturada, o Algoritmo BHT para resolução do problema da colisão e o Algoritmo de Shor para fatoração de inteiros, apoiados pelas sub-rotinas da Transformada de Fourier Quântica (QFT) e da Estimação de Fases Quântica (QPE). Os resultados validam o ganho de eficiência e as vantagens inerentes do processamento quântico frente aos métodos clássicos tradicionais.

Palavras-chave: Computação Quântica, Qiskit, Algoritmo de Shor, Algoritmo de Grover, Algoritmo BHT, Informação Quântica.

Abstract

This work presents a practical and theoretical study of quantum computing fundamentals and its main algorithms. The central objective is to bridge the gap between the mathematical formalism of Hilbert Space and the implementation of functional logic circuits. Using Python and the Qiskit framework, experiments were developed and simulated covering the creation of the four Bell States, the empirical verification of the CHSH Inequality, and multipartite entanglement (GHZ). Additionally, the research details the architecture and execution of advanced quantum algorithms, specifically Grover's Algorithm for unstructured search, the BHT Algorithm for the collision problem, and Shor's Algorithm for integer factorization, supported by the Quantum Fourier Transform (QFT) and Quantum Phase Estimation (QPE) subroutines. The results validate the efficiency gain and inherent advantages of quantum processing over traditional classical methods.

Keywords: Quantum Computing, Qiskit, Shor's Algorithm, Grover's Algorithm, BHT Algorithm, Quantum Information.

Sumário

Agradecimentos **5**

1 Introdução **6**
1.1 Motivações e objetivos 6
1.2 Ambientes de programação quântica 6

2 Fundamentos Matemáticos e Físicos **7**
2.1 Espaço de Hilbert 8
2.2 A Geometria do Qubit e a Natureza das fases 8
2.3 Fase Global vs. Fase Relativa 9
2.4 Operadores Unitários e Hermitianos 10
2.5 Aplicações 15
2.6 Portas lógicas quânticas 16
2.7 Circuitos Quânticos 17
2.8 Algoritmos Quânticos 20

3 Emaranhamento e Experimentos Práticos **20**
3.1 Implementação do primeiro Estado de Bell 21
3.2 Implementação do segundo Estado de Bell 21
3.3 Implementação do terceiro Estado de Bell 23
3.4 Implementação do quarto Estado de Bell 24
3.5 Desigualdade CHSH e a violação do Realismo Local 25
3.6 Implementação do Estado GHZ 28
3.7 Sub-rotinas e Operações controladas 28

4 Algoritmo de Grover **32**
4.1 O Oráculo Quântico (U_ω) 33
4.2 Gerenciamento de Ancillas e Descomputação 34
4.3 O Difusor e a reflexão de Householder 35
4.4 Implementação do Algoritmo de Grover 36
4.5 Complexidade do algoritmo de Grover 37

5 O Algoritmo BHT e o Problema da Colisão **38**
5.1 O Problema da Colisão em Funções 2-para-1 38
5.2 A Estratégia Híbrida e a Construção do Oráculo BHT 38
5.3 Implementação do Algoritmo BHT 39
5.4 Complexidade do Algoritmo BHT 41

6 Transformada de Fourier Quântica (QFT) **42**
6.1 A Falsa Equivalência e o Paradoxo da Medição 43
6.2 Implementação e a Representação em Produto 43
6.3 A Necessidade das Portas SWAP: Uma Prova Algébrica 44
6.4 Complexidade do Algoritmo da Transformada de Fourier Quântica (QFT) 45

7 Algoritmo de Estimação de Fases (QPE) **46**
7.1 A Mecânica do Phase Kickback 46
7.2 Codificação em Potências de U 47
7.3 A Transformada de Fourier Inversa como Decodificador 47
7.4 Implementação do Algoritmo de Estimação de Fases (QPE) . . 48
7.5 Complexidade do Algoritmo de Estimação de Fases (QPE) . . 50

8 Algoritmo de Shor **51**
8.1 Implementação do Algoritmo de Shor 52
8.2 Complexidade do Algoritmo de Shor 56

9 Pesquisas Futuras e Algoritmos de Consulta **57**

Agradecimentos

A **Deus**,

À minha **família**,

Ao meu orientador, **Prof. Dr. André Martin Timpanaro**,

Aos **monitores e colegas da UFABC**.

1 Introdução

1.1 Motivações e objetivos

O interesse em estudar a computação quântica decorre do potencial de processamento que emerge quando exploramos propriedades como superposição, emaranhamento e interferência. Contudo, a transição do formalismo matemático para a construção de tecnologia depende de uma compreensão profunda de como os operadores unitários manipulam os vetores no espaço de Hilbert.

Essa iniciação científica tem como motivação central o estudo e a implementação de alguns dos principais algoritmos quânticos. O foco não reside apenas na análise teórica, mas no desafio de traduzir o rigor matemático em circuitos funcionais. Ao implementar algoritmos fundamentais, como a Transformada de Fourier Quântica (QFT) e o algoritmo de Grover, busca-se investigar como o controle preciso de fases e rotações no plano complexo resulta em ganhos de eficiência computacional. Dessa forma, o trabalho visa consolidar o domínio sobre as ferramentas de simulação e programação quântica, estabelecendo uma base sólida para o estudo de problemas de maiores complexidades.

Para garantir a transparência e a reprodutibilidade desta pesquisa, todas as implementações práticas, códigos em Python (Qiskit), scripts de automação e cadernos interativos estão exaustivamente documentados e disponíveis para consulta pública. O repositório central do projeto pode ser acessado no GitHub [Lei26b], e as simulações executáveis estão hospedadas no ambiente Google Colab [Lei26a]. Ao longo das seções seguintes, serão apresentados no corpo do texto apenas os trechos de código estruturais e os diagramas lógicos essenciais para a compreensão teórica de cada algoritmo.

1.2 Ambientes de programação quântica

A transição da teoria quântica para a prática computacional exige ferramentas de software capazes de abstrair a complexidade algébrica dos tensores e espaços de Hilbert em instruções de programação modulares. Neste contexto, o Qiskit, um *framework* de código aberto desenvolvido pela IBM em Python, consolida-

se como um dos principais ambientes de desenvolvimento quântico [IBM26]. A adoção da linguagem Python facilita a integração nativa dos circuitos quânticos com ecossistemas consolidados de análise de dados, inteligência artificial e visualização científica. O Qiskit permite a construção, simulação e execução de algoritmos por meio de abstrações como a classe `QuantumCircuit`. Além disso, a biblioteca oferece simuladores locais de alto desempenho, como o `AerSimulator`, e fornece a infraestrutura necessária para submeter *jobs* diretamente a processadores quânticos reais (QPUs) baseados em circuitos supercondutores através da nuvem.

2 Fundamentos Matemáticos e Físicos

No plano complexo, um número $Z = a + bi$ é representado como um ponto em um sistema de coordenadas onde o eixo horizontal descreve a parte real (a) e o eixo vertical a parte imaginária (b). Para o estudo de sistemas quânticos, é mais conveniente utilizar a forma polar (r, ϕ), onde r representa a magnitude (raio) e ϕ a fase (ângulo). As relações de conversão são dadas por:

$$a = r\cos\phi \tag{1}$$

$$b = r\sin\phi \tag{2}$$

Dessa forma, Z assume a forma trigonométrica $Z = r(\cos\phi + i\sin\phi)$. Através da fórmula de Euler, $e^{i\phi} = \cos\phi + i\sin\phi$, podemos simplificar essa expressão para a forma exponencial $Z = re^{i\phi}$.

Quando restringimos o raio ao valor unitário ($r = 1$), os números complexos resultantes mapeiam o círculo unitário. Este conceito é o alicerce para a representação de estados na **Esfera de Bloch** (vide Figura 1). Enquanto um número complexo unitário descreve uma trajetória em um círculo bidimensional, um qubit é representado na superfície de uma esfera tridimensional. Nela, a conservação de probabilidade exige que o estado seja normalizado $\langle\psi \mid \psi\rangle = 1$. No mapeamento da esfera, o ângulo θ define a probabilidade de colapso entre os estados da base computacional $|0\rangle$ e $|1\rangle$, enquanto ϕ caracteriza a fase relativa entre eles.

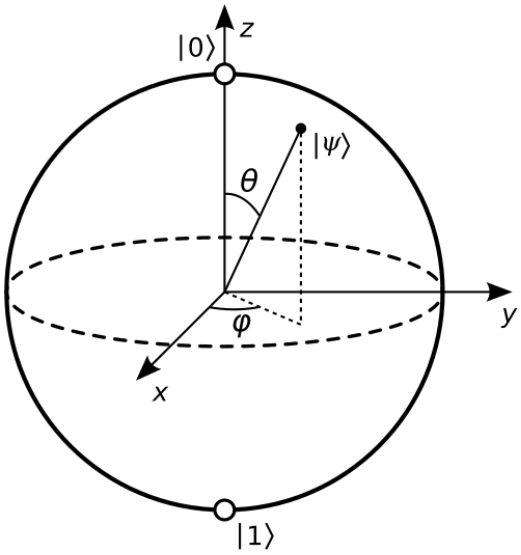


Figura 1: A esfera de Bloch, uma representação geométrica de um sistema quântico de dois níveis [SM07].

2.1 Espaço de Hilbert

A descrição de sistemas quânticos fundamenta-se na estrutura do Espaço de Hilbert ($\mathcal{H}$), um espaço vetorial complexo completo munido de um produto interno. Enquanto a computação clássica baseia-se em estados discretos e determinísticos, um qubit é representado como um vetor unitário neste espaço. A importância de ($\mathcal{H}$) reside na sua métrica: o produto interno permite definir a ortogonalidade entre estados — essencial para distinguir os estados da base $|0\rangle$ e $|1\rangle$ — e calcular amplitudes de probabilidade através da regra de Born. Além disso, a característica linear deste espaço é o que sustenta o princípio da superposição, permitindo que a informação quântica exista em combinações lineares que não possuem análogo clássico.

2.2 A Geometria do Qubit e a Natureza das fases

Na representação da Esfera de Bloch (Figura 1), os polos definem a base computacional: o Polo Norte corresponde ao estado $|0\rangle$ e o Polo Sul ao estado $|1\rangle$. Qualquer estado puro de um qubit pode ser visualizado como um vetor tridimensional que aponta do centro para a superfície da esfera.

Para visualizar a superposição, pode-se traçar um paralelo com uma moeda girando no ar. Enquanto uma moeda pousada sobre a mesa representa um bit clássico (estritamente cara ou coroa), a moeda em rotação representa o qubit: ela não está em nenhum dos dois estados absolutos, mas sim em um espectro contínuo de probabilidades. É apenas no momento em que a moeda é parada por uma mão — o equivalente à medição quântica — que o sistema é forçado a colapsar para um valor definido.

Matematicamente, utilizando os ângulos de coordenadas esféricas, o estado geral de um qubit é rigorosamente definido pela equação:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle \tag{3}$$

Uma característica contra-intuitiva dessa formulação é a divisão do ângulo polar θ por 2. Essa adaptação é uma necessidade geométrica profunda: no Espaço de Hilbert abstrato, os estados $|0\rangle$ e $|1\rangle$ são vetores ortogonais (separados por 90°). Contudo, na visualização 3D da esfera de Bloch, eles ocupam polos opostos (separados por 180°). O fator $\theta/2$ atua como um conversor entre a álgebra linear complexa e a geometria euclidiana tridimensional.

A inclinação do vetor, dada pela latitude θ (onde $0 \leq \theta \leq \pi$), governa estritamente a **probabilidade** de colapso durante uma medição, obedecendo à regra de Born. Se o vetor repousa exatamente no equador da esfera ($\theta = \pi/2$), as probabilidades de medir 0 ou 1 tornam-se perfeitamente iguais (50% para cada).

2.3 Fase Global vs. Fase Relativa

O termo $e^{i\phi}$ na equação 3 introduz o conceito de fase, que deve ser cuidadosamente dividido em duas categorias:

1. **Fase Global:** Representa um deslocamento de fase aplicado uniformemente a todo o vetor de estado, como em $|\psi'\rangle = e^{i\gamma}|\psi\rangle$. Fisicamente, a fase global é indetectável. Como a probabilidade de medição depende do módulo ao quadrado das amplitudes, e o módulo de $e^{i\gamma}$ é sempre 1, um estado multiplicado por uma fase global possui as exatas mesmas

estatísticas de medição que o estado original. Por esta razão, ela é comumente descartada nos cálculos algébricos de circuitos quânticos.

2. **Fase Relativa (ϕ):** Refere-se à diferença de fase geométrica entre as amplitudes da superposição. Na esfera de Bloch, o ângulo ϕ (onde $0 \leq \phi < 2\pi$) representa o movimento do vetor ao longo da longitude, ou seja, ao redor do equador.

A fase relativa é o "motor"da computação quântica. Alterar a fase ϕ de um vetor que está no equador não muda suas probabilidades imediatas de colapso para 0 ou 1. No entanto, ela determina a direção para a qual o vetor aponta no plano imaginário. É essa orientação relativa que permite que as amplitudes se somem (interferência construtiva) ou se cancelem (interferência destrutiva) ao passarem por portas lógicas subsequentes. Algoritmos como o de Grover e a Transformada de Fourier Quântica (QFT) são construídos fundamentalmente sobre a manipulação cirúrgica da fase relativa para extrair respostas de superposições massivas.

Para ilustrar, se multiplicamos o estado geral por um fator $e^{i\delta}$, estamos rotacionando o vetor geometricamente ao longo do eixo Z da esfera, preservando suas probabilidades originais, mas preparando-o para diferentes padrões de interferência em passos computacionais futuros.

2.4 Operadores Unitários e Hermitianos

Os operadores são as ações fundamentais aplicadas sobre os qubits. São transformações lineares que atuam no espaço de Hilbert. Para um sistema de n qubits, esses operadores são representados por matrizes, onde cada entrada da matriz descreve como a amplitude de um estado base é transferida para outro.

Operadores unitários são as ações, ou portas lógicas, usados para construir os algoritmos quânticos. Também representam a evolução temporal de um sistema quântico fechado. Matematicamente, preservam o produto interno, o que garante que a norma do vetor de estado (e, portanto, a probabilidade total)

permaneça inalterada após a operação e a propriedade $U^\dagger U = I$ também garante que a operação seja reversível.

Por outro lado, os operadores hermitianos são usados para medir e obter o resultado final de um qubit após uma sequência de operações sobre ele. Correspondem aos observáveis físicos. Sua principal característica é possuir autovalores reais, o que é consistente com o fato de que medições laboratoriais resultam em valores reais, e não em complexos.

A seguinte tabela apresenta os principais conceitos que formam a base da teoria quântica, a partir da notação de Dirac.

Notação	Descrição
Z^*	Complexo conjugado do número complexo Z. $(1+i)^* = 1-i$.
$\|\psi\rangle$	Um vetor em um espaço vetorial abstrato complexo. Representa um estado em algum sistema quântico. Conhecido como Ket.
$\langle\psi\|$	O vetor dual de $\|\psi\rangle$. Conhecido como Bra.
$\langle\phi \mid \psi\rangle$	Produto interno entre $\|\phi\rangle$ e $\|\psi\rangle$
$\|\phi\rangle \otimes \|\psi\rangle$	Produto tensorial de $\|\phi\rangle$ e $\|\psi\rangle$. Também abreviado para $\|\phi\rangle\|\psi\rangle$
A^*	Conjugado complexo da matriz A
A^T	Matriz transposta da matriz A
$A^\dagger$	Conjugado hermitiano (adjunto) da matriz A. Temos que $A^\dagger = (A^T)^*$. $\begin{bmatrix} a & b \\ c & d \end{bmatrix}^\dagger = \begin{bmatrix} a^* & c^* \\ b^* & d^* \end{bmatrix}$

Tabela 1: Algumas notações da mecânica quântica a partir da álgebra linear. Adaptado de [NC10]

Na notação de Dirac, o Bra e o Ket representam o mesmo estado em espaços vetoriais duais. Enquanto o Ket é representado por um vetor coluna, o Bra é representado por um vetor linha, sendo obtido pela transposta conjugada do Ket.

Exemplo:

$$|\phi\rangle = \frac{3}{5}|0\rangle + \frac{4i}{5}|1\rangle = \begin{pmatrix} \frac{3}{5} \\ \\ \frac{4i}{5} \end{pmatrix}$$

Ao transpor, é obtido

$$|\phi\rangle^T = \begin{pmatrix} \frac{3}{5} & \frac{4i}{5} \end{pmatrix}^T$$

Pelo conjugado complexo, tem-se

$$(\begin{pmatrix} \frac{3}{5} & \frac{4i}{5} \end{pmatrix}^T)^* = \begin{pmatrix} \frac{3}{5} & -\frac{4i}{5} \end{pmatrix}$$

Finalmente,

$$\langle\phi| = \frac{3}{5}\langle 0| - \frac{4i}{5}\langle 1|$$

O Bra é comumente usado para calcular o produto interno entre dois vetores. A operação $\langle\phi \mid \phi\rangle$ resulta em um escalar, se tornando uma base para calcular probabilidades, projeções e inúmeras aplicações em mecânica quântica.

Um **espaço vetorial primal** contém os objetos fundamentais do sistema, como os Kets $|\psi\rangle$. Eles representam o estado físico atual e 'moram' em um espaço complexo V.

O **espaço dual** V^* contém os funcionais lineares, representados pelos Bras $\langle\psi|$. É o conjunto de todas as funções que mapeiam um vetor do espaço primal para um escalar complexo. Essa relação é o que permite que o produto interno $\langle\psi \mid \psi\rangle$ resulte em um número real.

$$\langle\phi \mid \psi\rangle = \begin{pmatrix} \gamma^* & \delta^* \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \gamma^*\alpha + \delta^*\beta$$

O **qubit** da computação quântica é o equivalente ao bit do computador clássico. Diferente de um bit clássico, a estrutura de espaço vetorial de $\mathcal{H}$ permite que um qubit exista em uma **combinação linear** de seus estados de base. Assim, um estado genérico é escrito como $|\phi\rangle = \alpha|0\rangle + \beta|1\rangle$, com $\alpha, \beta \in \mathbb{C}$ chamados de amplitude de probabilidade. As amplitudes podem ser negativas ou complexas. É essa característica que permite a **interferência** (amplitudes que se cancelam), algo que não existe na probabilidade clássica. Matematicamente, $|0\rangle$ e $|1\rangle$ são vetores ortogonais. Mas, fisicamente, eles representam dois estados distinguíveis e mensuráveis de um sistema quântico real, como o spin de um elétron, polarização de um fóton, níveis de energia de um átomo, etc. Embora a base computacional ($|0\rangle, |1\rangle$) seja a mais comum, o qubit pode ser representado em outras bases, como a base de Hadamard ($|+\rangle, |-\rangle$). Para calcular a probabilidade de medir o qubit como 0 ou 1,

calcula-se o módulo ao quadrado da sua amplitude correspondente.

Exemplo: A probabilidade de medir $|0\rangle$ é $P(0) = |\alpha|^2$, enquanto a probabilidade de medir $|1\rangle$ é $P(1) = |\beta|^2$.

A regra da normalização é que a soma de todas as probabilidades seja $|\alpha|^2 + |\beta|^2 = 1$. Um qubit estar em superposição perfeita, com probabilidade igual de ser medido como 0 ou 1, $P(0) = P(1) = \frac{1}{2}$, significa que $|\alpha|^2 = \frac{1}{2} \longrightarrow |\alpha| = \frac{1}{\sqrt{2}} = |\beta|$. Portanto, o estado

$$|+\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle \tag{4}$$

obedece à regra da normalização. Não por acaso, o estado definido em (4) representa um qubit na base de Hadamard.

Para realizar cálculos mais complexos e programas mais sofisticados, faz-se necessário utilizar mais de um qubit em um circuito quântico. Para isso, utiliza-se o **Produto tensorial ($\otimes$)** que permite:

1. Descrever o espaço de Hilbert do sistema composto, que são sistemas de múltiplas partes, com mais de 1 qubit ou interações entre elétrons.

2. Representar operações conjuntas sobre múltiplos qubits. Ou seja, pode-se representar um espaço de N qubits que possui 2^N dimensões.

3. Combinar estados de sistemas quânticos independentes. Isto é, no espaço combinado de 2 qubits, existem estados que não podem ser escritos como um produto tensorial simples de estados individuais. Um exemplo é o estado de Bell $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ que não pode ser fatorado em $(a|0\rangle + b|1\rangle) \otimes (c|0\rangle + d|1\rangle)$ com $a, b, c, d \in \mathbb{C}$. Estes estados "não fatoráveis" são os **estados emaranhados**. Assim, definindo e identificando o **emaranhamento**, que não está presente em sistemas individuais.

O emaranhamento quântico funciona como um par de dados perfeitamente sincronizados. Se dois dados clássicos são lançados em locais distintos, os resultados são estatisticamente independentes. Contudo, se esses dados estivessem quanticamente emaranhados, ao observar que o primeiro dado

resultou no número 6, o observador saberia instantaneamente que o segundo dado, mesmo a anos-luz de distância, também resultou no número 6. Essa correlação instantânea, que desafia a intuição espacial clássica está por trás do funcionamento de muitos algoritmos quânticos.

Exemplo: $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, |0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

$$|10\rangle = |1\rangle \otimes |0\rangle = \begin{pmatrix} 0 \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ 1 \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

Exemplo: Para aplicar uma porta em um qubit e não afetar a outro, podemos usar a porta Pauli-X no primeiro e a Identidade I no segundo: $X \otimes I$.

O **conjugado complexo** de uma matriz é a matriz obtida calculando o conjugado complexo de cada elemento individualmente sem mudar a posição de ninguém. O conjugado de $a + bi$ é $a - bi$. De $a - bi$ é $a + bi$. O conjugado complexo de um número real é ele mesmo.

A matriz **transposta conjugada** ou **adjunta hermitiana** é uma operação muito mais importante e comum em mecânica quântica, pois é ela que define os operadores hermitianos ($A = A^\dagger$) e unitários ($U^\dagger = U^{-1}$).

O dual de um Ket $|\psi\rangle$ (vetor coluna) é o Bra $\langle\psi|$ (vetor linha conjugado). A operação que faz essa conversão é a operação Dagger $|\psi\rangle^\dagger = \langle\psi|$.

A operação Dagger faz a mesma coisa para os operadores. Ou seja, se um operador A age em um Ket, o resultado é um novo Ket $A|\psi\rangle$. Se quisermos saber qual é o Bra correspondente, basta fazer $(A|\psi\rangle)^\dagger = \langle\psi|A^\dagger$. Onde a operação Dagger "traduziu" o Ket para o Bra e o operador A para $A^\dagger$. Isso nos mostra como um operador se comporta no espaço dual.

Essa operação é a ferramenta matemática que nos permite criar operadores que forçam o cumprimento dessas duas regras:

1. Os resultados de uma medição no mundo real são sempre números reais.

2. A probabilidade total de um sistema existir tem que ser 100% (ou 1). Uma operação não pode fazer a probabilidade desaparecer ou exceder 1.

Um operador é **hermitiano** se e somente se ele é sua própria tradução para o espaço dual, ou seja, $A = A^\dagger$.

Essa propriedade nos garante que os autovalores do operador (os possíveis resultados de uma medição) são números reais.

A dagger funciona como um "teste de realidade". Se um operador representa uma quantidade física que podemos medir (energia, posição, spin do elétron), ele precisa ser hermitiano, pois o resultado de uma medição no laboratório é sempre um número real.

Um operador é **unitário** se a sua tradução para o espaço dual é igual à sua operação inversa, ou seja, $U^\dagger = U^{-1}$.

Essa propriedade garante que o operador conserva o comprimento do vetor de estado. Como esse comprimento ao quadrado é a probabilidade total (100%), a dagger aqui funciona como um "teste de conservação". Se um operador é unitário, ele representa uma evolução física e reversível, ou seja, uma porta lógica quântica. Ele embaralha as probabilidades, mas nunca as cria ou destrói. Todas as portas lógicas são, por definição, unitárias, pois elas manipulam a informação sem violar a lei de conservação de probabilidade.

2.5 Aplicações

A regra fundamental de que um estado quântico deve ser normalizado, $\langle\psi \mid \psi\rangle = 1$, é na verdade $(|\psi\rangle^\dagger)|\psi\rangle = 1$. A dagger está na própria definição de probabilidade.

Para saber o valor médio de uma medição do operador A em um estado $|\psi\rangle$, deve-se calcular $\langle\psi|A|\psi\rangle$. Novamente, a dagger está em si, definindo o bra.

Circuitos quânticos, portas lógicas, o algoritmo de Grover e o algoritmo de Shor, por exemplo, são construídos sobre a manipulação de estados através de operadores unitários e a extração de informações através de medições descritas por operadores hermitianos.

Um operador A é como uma transformação no espaço (girar, esticar, espelhar). Existem alguns vetores que não mudam de direção quando tais transformações são aplicadas. Tais são chamados de **Autovetores**. O número

que diz o quanto o autovetor foi transformado é chamado de **Autovalor**, sendo o fator de escala.

A relação é descrita: $A|v\rangle = \lambda|v\rangle$. Aplicar o operador A no seu autovetor $|v\rangle$ tem o mesmo efeito que multiplicar $|v\rangle$ pelo autovalor λ (que é um escalar).

Os autovetores de um operador de medição representam os únicos estados definidos em que o sistema pode estar após a medição.

Pensando na base computacional $|0\rangle$ e $|1\rangle$, tais são os autovetores do operador de medição Pauli-Z (que mede o spin na direção Z). Isso significa que, não importa qual seja a superposição inicial do qubit, quando é medido nesta base, ele é forçado a escolher uma das direções (autovetores). O resultado tem que ser ou $|0\rangle$ ou $|1\rangle$.

O autovalor associado a cada autovetor é o valor numérico real que a medição registra. Logo, se o autorvetor é o estado final, o autovalor é o número anotado. Por exemplo, para o operador de medição Pauli-Z, o autovetor $|0\rangle$ tem o autovalor $+1$, enquanto que o autovetor $|1\rangle$ tem o autovalor -1.

2.6 Portas lógicas quânticas

São os operadores unitários usados para executar os passos de uma computação quântica. Dentre as principais temos:

Porta X (Pauli-X): é a porta NOT quântica que age nos estados $|0\rangle$ e $|1\rangle$ da mesma forma que a porta NOT clássica age nos bits 0 e 1:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \tag{5}$$

$$X|0\rangle \longrightarrow |1\rangle \tag{6}$$

$$X|1\rangle \longrightarrow |0\rangle \tag{7}$$

Porta H (Hadamard): é a porta da superposição. Transforma estados definidos em superposições simétricas:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \tag{8}$$

$$H|0\rangle \longrightarrow |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \tag{9}$$

$$H|1\rangle \longrightarrow |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \tag{10}$$

Portas Z e S (portas de fase): não mudam as probabilidades de medir 0 ou 1, mas introduzem uma fase relativa, algo crucial para a interferência quântica.

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad \text{e} \quad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} \tag{11}$$

Porta CNOT (controlled not): esta é a primeira porta que faz os qubits interagirem, sendo a chave para criar o **emaranhamento**. Ela aplica uma porta NOT (X) no segundo qubit (alvo) se, e somente se, o qubit de controle estiver no estado $|1\rangle$.

$$CNOT = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \tag{12}$$

2.7 Circuitos Quânticos

No geral, os algoritmos quânticos podem ser implementados através de circuitos quânticos. Esses circuitos são abstrações gráficas onde as linhas horizontais representam a evolução dos qubits no tempo, e os blocos representam a aplicação sucessiva de operadores unitários. A ordem de leitura de um circuito é intuitiva, mas é importante lembrar que, na álgebra, a aplicação de portas A,

B e C em sequência sobre um estado $|\psi\rangle$ é escrita da direita para a esquerda: $C \cdot B \cdot A|\psi\rangle$.

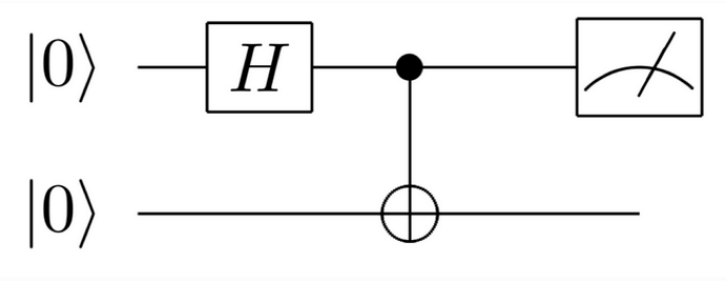


Figura 2: Exemplo de diagrama de circuito quântico. [Qul24]

Cada linha horizontal representa um qubit. O tempo flui da esquerda para a direita. As "caixas" nas linhas são as portas lógicas (nesse caso a porta Hadamard). As linhas verticais conectando as linhas dos qubits representam portas de múltiplos qubits. No final, um símbolo de medição indica a leitura do resultado (operador hermitiano). Logo o exemplo da figura 2 pode ser descrito como: Tomar um qubit q_0, aplicar a porta Hadamard para criar a superposição, usar uma porta CNOT para emaranhá-lo com q_1 e finalmente medir q_0.

Ler um circuito da esquerda para a direita é o mesmo que multiplicar as matrizes das portas da direita para a esquerda no seu estado inicial, ou multiplicar a matriz da porta pelo vetor de estado do qubit. Logo, o estado final $|\psi_{final}\rangle$ antes da medição na figura 2 é $CNOT \cdot (H \otimes I) \cdot |\psi_{inicial}\rangle$.

Como nesse caso estamos lidando com 2 qubits, o espaço de Hilbert tem 4 dimensões e é obtido através do produto tensorial, de forma que a base computacional fica $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. Vamos então determinar explicitamente $|\psi_{final}\rangle$:

Primeiro, começamos com dois qubits q_0 e q_1 no estado $|0\rangle$. Usando o produto tensorial, o estado combinado do sistema se torna $|\psi_0\rangle = |0\rangle \otimes |0\rangle = |00\rangle$.

Segundo, aplicamos a porta H em q_0 e a porta I em q_1. O operador

combinado é $H \otimes I$.

$$|\psi_1\rangle = (H \otimes I)|\psi_0\rangle = (H|0\rangle) \otimes (I|0\rangle) \tag{13}$$

$$|\psi_1\rangle = |+\rangle \otimes |0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |0\rangle \tag{14}$$

Distribuindo o produto tensorial, temos:

$$|\psi_1\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) \tag{15}$$

Agora aplicamos a operação $CNOT$ no estado $|\psi_1\rangle$:

$$|\psi_2\rangle = CNOT|\psi_1\rangle = CNOT\left(\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)\right) \tag{16}$$

O $CNOT$ age em cada parte da superposição: no termo $|00\rangle$ o qubit de controle q_0 é 0, então nada acontece ($|00\rangle$). No termo $|10\rangle$ o qubit de controle q_0 é 1, então o $CNOT$ inverte o q_1 de 0 para 1 ($|11\rangle$). O estado final é então o chamado estado de Bell:

$$|\psi_{final}\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \tag{17}$$

O resultado (17) mostra o emaranhamento criado. Se medirmos q_0 e obtivermos 0, q_1 será com 100% de certeza 0. Se medirmos q_0 como 1, q_1 será 1. As partículas estão conectadas.

Agora, o ato de medir é destrutivo (com respeito à superposição) e descrito por um operador hermitiano. Nós sempre medimos com respeito a uma base, sendo a mais comum delas a computacional.

Existem 2 regras que governam o que acontece durante a medição, que são postulados da mecânica quântica.

Regra 1: A regra de Born (calculando as probabilidades). Esta regra nos diz quais são as chances de obter cada resultado possível.

Definição: para um estado quântico geral $|\psi\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle + ...$, a probabilidade de medir um estado base específico (ex: $|0\rangle$) é o módulo ao quadrado da sua amplitude: $P(0) = |\alpha_0|^2$, $P(1) = |\alpha_1|^2$, etc. Isso nos diz que,

ao medir o estado de Bell, nunca será obtido 01 ou 10. O emaranhamento é refletido nas probabilidades.

Regra 2: o colapso da função de onda (o estado pós-medição). Imediatamente após a medição projetiva resultar em um autoestado específico $|i\rangle$, o sistema colapsa para $|i\rangle$, de modo que medições subsequentes na mesma base retornarão deterministicamente o mesmo resultado.

2.8 Algoritmos Quânticos

Diferente dos algoritmos clássicos que operam de forma determinística sobre bits em estados definidos, os algoritmos quânticos exploram as propriedades fundamentais da mecânica quântica - como a superposição, o emaranhamento e a interferência - para realizar transformações lineares complexas em um único passo computacional [Abh+22]. A verdadeira vantagem quântica manifesta-se em classes de problemas específicos onde operadores unitários são orquestrados para amplificar as probabilidades de respostas corretas (interferência construtiva) e suprimir as incorretas (interferência destrutiva). Esta manipulação das amplitudes de probabilidade é o mecanismo central que proporciona acelerações polinomiais (como na busca em bases não estruturadas) ou exponenciais (como na fatoração de inteiros) em relação aos melhores algoritmos clássicos conhecidos.

3 Emaranhamento e Experimentos Práticos

No que se segue, todos os códigos estão precedidos do seguinte cabeçalho, que será omitido para manter as coisas mais compactas

```
import numpy as np
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.circuit.library import UnitaryGate
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
```

Código 1: Cabeçalho básico para todos os códigos usando Qiskit

3.1 Implementação do primeiro Estado de Bell

Como um exemplo de emaranhamento, implementamos o estado $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ utilizando a biblioteca Qiskit. O código abaixo detalha cada etapa, desde a criação do circuito até a simulação estatística.

```
# 1. Definição do circuito (2 qubits e 2 bits clássicos)
circuito = QuantumCircuit(2, 2)

# 2. Criação de Superposição (porta Hadamard)
circuito.h(0)

# 3. Emaranhamento quântico (porta CNOT)
circuito.cx(0, 1)

# 4. Medição (colapso da função de onda)
circuito.measure([0, 1], [0, 1])
```

Código 2: Implementação em Python para a geração do Primeiro Estado de Bell $|\Phi^+\rangle$. [Gru26]

O circuito gerado pelo código pode ser visualizado na Figura 3.

Ao executar a simulação, obtivemos resultados concentrados nos estados $|00\rangle$ e $|11\rangle$ (aproximadamente 50% cada). A ausência dos estados $|01\rangle$ e $|10\rangle$ confirma o emaranhamento. Ou seja, a medição de um qubit determina o estado do outro.

3.2 Implementação do segundo Estado de Bell

Para a geração do estado $|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle)$, introduzimos uma inversão de fase no sistema. Diferente do estado anterior, aplicamos uma porta X no qubit de controle antes da porta Hadamard, garantindo que a superposição ocorra a partir do estado de base $|1\rangle$.

```
circuito = QuantumCircuit(2, 2)

# 1. Invertemos o qubit de controle para |1>
circuito.x(0)
```

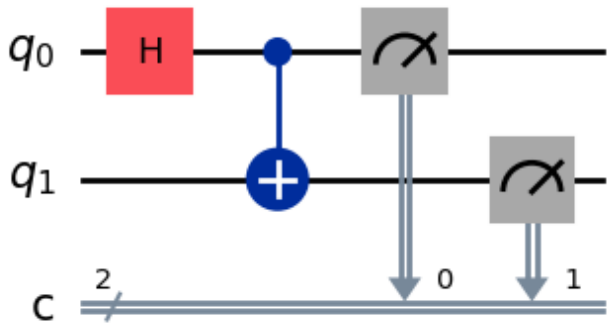


Figura 3: Circuito lógico do estado $|\Phi^+\rangle$ gerado via Qiskit, exibindo as portas H e $CNOT$ seguidas pelos operadores de medição.

```
5
6  # 2. Superposição com fase relativa
7  circuito.h(0)
8
9  # 3. Emaranhamento quântico (porta CNOT)
10 circuito.cx(0, 1)
11
12 # 4. Medição
13 circuito.measure([0, 1], [0, 1])
```

Código 3: Implementação em Python para a geração do Segundo Estado de Bell $|\Phi^-\rangle$.

O circuito resultante, apresentando a porta de pré-processamento X, pode ser visualizado na Figura 4.

Embora o histograma de medição apresente resultados estatisticamente idênticos ao estado $|\Phi^+\rangle$ (distribuição equilibrada entre $|00\rangle$ e $|11\rangle$), a diferença reside na fase relativa entre os estados e se medissemos outra base que não a computacional, a estatística mudaria.

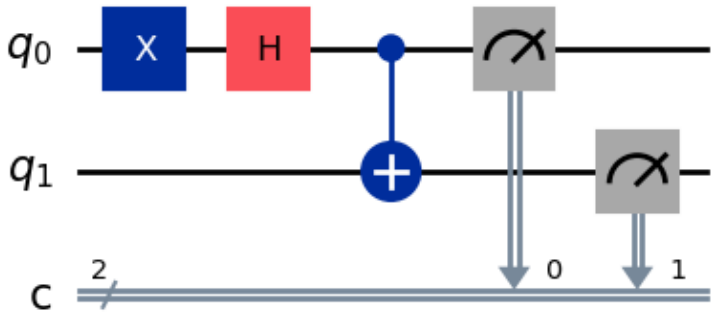


Figura 4: Circuito lógico do estado $|\Phi^-\rangle$, evidenciando a porta Pauli-X inicial utilizada para ajuste de fase.

3.3 Implementação do terceiro Estado de Bell

O estado $|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle)$ caracteriza-se pela anti-correlação perfeita. Para implementá-lo, invertemos o qubit alvo ($|q_1\rangle$) para o estado $|1\rangle$ antes da aplicação da porta $CNOT$, alterando a paridade do resultado final.

```
circuito = QuantumCircuit(2, 2)

# 1. Invertemos o qubit alvo para |1>
circuito.x(1)

# 2. Superposição no qubit de controle
circuito.h(0)

# 3. Emaranhamento quântico (porta CNOT)
circuito.cx(0, 1)

# 4. Medição
circuito.measure([0, 1], [0, 1])
```

Código 4: Implementação em Python para a geração do Terceiro Estado de Bell $|\Psi^+\rangle$.

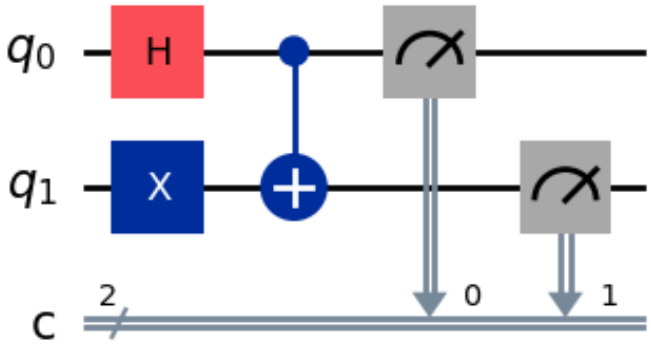


Figura 5: Circuito lógico do estado $|\Psi^+\rangle$, representando a preparação de uma superposição simétrica entre os estados $|01\rangle$ e $|10\rangle$.

Neste cenário, a simulação demonstra que, ao medir $|0\rangle$ no primeiro qubit, o segundo invariavelmente colapsará em $|1\rangle$, e vice-versa. Os dados obtidos confirmam que apenas os estados $|01\rangle$ e $|10\rangle$ são medidos.

3.4 Implementação do quarto Estado de Bell

Finalmente, é implementado o estado $|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$. Este estado combina a anti-correlação de bits com a inversão de fase, exigindo a manipulação de ambos os qubits antes do emaranhamento.

```
circuito = QuantumCircuit(2, 2)

# 1. Invertemos ambos os qubits para |11>
circuito.x(0)
circuito.x(1)

# 2. Superposição com fase no qubit de controle
circuito.h(0)

# 3. Emaranhamento quântico (porta CNOT)
circuito.cx(0, 1)
```

```
12
13 # 4. Medição
14 circuito.measure([0, 1], [0, 1])
```

Código 5: Implementação em Python para a geração do Quarto Estado de Bell $|\Psi^-\rangle$.

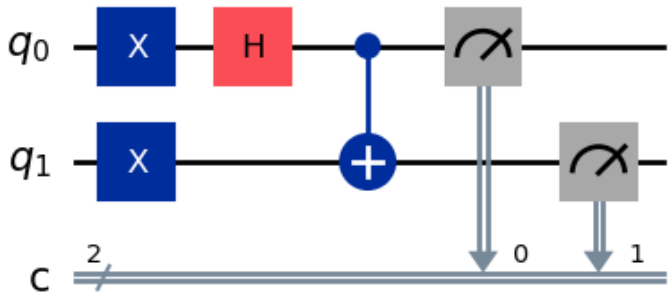


Figura 6: Circuito lógico do estado $|\Psi^-\rangle$, configurado para gerar estados anti-simétricos com oposição de fase.

A análise dos quatro estados implementados demonstra a versatilidade do Qiskit na manipulação do espaço de Hilbert. A transição entre correlação ($|\Phi\rangle$) e anti-correlação ($|\Psi\rangle$) é controlada de forma determinística através de portas de bit-flip (X), enquanto a fase é modulada pela preparação da base antes da porta de Hadamard.

3.5 Desigualdade CHSH e a violação do Realismo Local

A criação matemática dos estados de Bell demonstra o fenômeno do emaranhamento, mas a comprovação de que essas correlações não podem ser explicadas por teorias clássicas de variáveis ocultas locais é dada pela desigualdade CHSH (Clauser-Horne-Shimony-Holt) [Cla+69], uma generalização do Teorema de Bell.

O teste CHSH considera duas partículas emaranhadas distribuídas para dois observadores (Alice e Bob). Eles realizam medições em bases rotacionadas

independentes. Alice escolhe entre as bases A e A', enquanto Bob escolhe entre B e B'. A correlação entre as medições é dada pelo valor esperado $E(\theta_a, \theta_b)$. O parâmetro S de CHSH é definido como:

$$S = E(A, B) - E(A, B') + E(A', B) + E(A', B') \tag{18}$$

Segundo a física clássica e o princípio do realismo local [NC10], o valor absoluto deste parâmetro está estritamente limitado por $|S| \leq 2$. No entanto, a mecânica quântica prevê que, ao medir um estado maximamente emaranhado (como o $|\Phi^+\rangle$) em ângulos específicos - tipicamente 0 e $\pi/2$ para Alice, e $\pi/4$ e $-\pi/4$ para Bob -, o limite clássico é violado, podendo alcançar o limite de Tsirelson de $S = 2\sqrt{2} \approx 2.828$.

```
def construir_circuito_chsh(ang_alice, ang_bob):
    """Cria um par emaranhado de Bell e gira os
   detectores de Alice e Bob nos ângulos especificados."
   ""
    circuito = QuantumCircuit(2, 2)
    # 1. Prepara Estado de Bell |Phi+>
    circuito.h(0)
    circuito.cx(0, 1)
    # 2. Rotações das bases de medição (simula os
   detectores girando no espaço)
    circuito.ry(ang_alice, 0)
    circuito.ry(ang_bob, 1)
    circuito.measure([0, 1], [0, 1])
    return circuito

# Ângulos ótimos que maximizam a quebra quântica
ang_A  = np.pi/2
ang_A_ = 0.
ang_B  = np.pi / 4
ang_B_ = -np.pi / 4

# 4 combinações de medições entre Alice e Bob
```

```
circuitos_chsh = [
    construir_circuito_chsh(ang_A , ang_B),
    construir_circuito_chsh(ang_A , ang_B_),
    construir_circuito_chsh(ang_A_, ang_B),
    construir_circuito_chsh(ang_A_, ang_B_)]

simulador = AerSimulator()
correlacoes_E = []

for circ in circuitos_chsh:
    contagens = simulador.run(transpile(circ, simulador), shots=8192).result().get_counts()
    iguais = contagens.get("00", 0) + contagens.get("11", 0)
    diferentes = contagens.get("01", 0) + contagens.get("10", 0)
    total = iguais + diferentes
    # Correlação E = P(iguais) - P(diferentes)
    E = (iguais - diferentes) / total
    correlacoes_E.append(E)

# Parâmetro CHSH: S = E(A,B)-E(A,B')+E(A',B)+E(A',B')
parametro_S = correlacoes_E[0] - correlacoes_E[1] + correlacoes_E[2] + correlacoes_E[3]
print(f"Parâmetro S medido: {parametro_S:.4f}")
```

Código 6: Implementação do teste experimental da Desigualdade CHSH simulando a rotação de bases independentes.

Através da simulação utilizando o `AerSimulator`, com 8192 *shots* para mitigar a variância estatística, obtivemos sistematicamente valores de S próximos de $2\sqrt{2}$. Este resultado simula com êxito a quebra do limite clássico, confirmando a natureza não-local do emaranhamento.

3.6 Implementação do Estado GHZ

Expandindo os conceitos de emaranhamento explorados nos estados de Bell, o estado GHZ (Greenberger-Horne-Zeilinger) [GHZ89] representa o emaranhamento de três ou mais subsistemas. Para um sistema de três qubits, o estado é definido como $|GHZ\rangle = \frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$.

A construção deste estado segue uma lógica análoga à do estado $|\Phi^+\rangle$. Inicia-se aplicando uma porta Hadamard ao primeiro qubit, criando uma superposição inicial. Em seguida, utilizam-se as portas CNOT em cascata para propagar esse estado para os qubits adjacentes.

```
circuito_ghz = QuantumCircuit(3, 3)

# 1. Superposição inicial no qubit 0
circuito_ghz.h(0)

# 2. Emaranhamento em cascata (portas CNOT)
# Qubit 0 emaranha com o qubit 1:
circuito_ghz.cx(0, 1)
# Qubit 1 (já emaranhado) emaranha com o qubit 2:
circuito_ghz.cx(1, 2)

# 3. Medição dos 3 qubits
circuito_ghz.measure([0, 1, 2], [0, 1, 2])
```

Código 7: Implementação em Python para a geração do Estado GHZ com emaranhamento multipartido.

Ao realizar a simulação deste circuito, os resultados de medição colapsam exclusivamente para os estados $|000\rangle$ e $|111\rangle$, evidenciando o emaranhamento multipartido onde a medição de um único qubit determina o estado de todo o sistema.

3.7 Sub-rotinas e Operações controladas

O desenvolvimento de algoritmos quânticos mais complexos, como o algoritmo de Grover (que será desenvolvido na próxima seção) [NC10], exige a aplicação

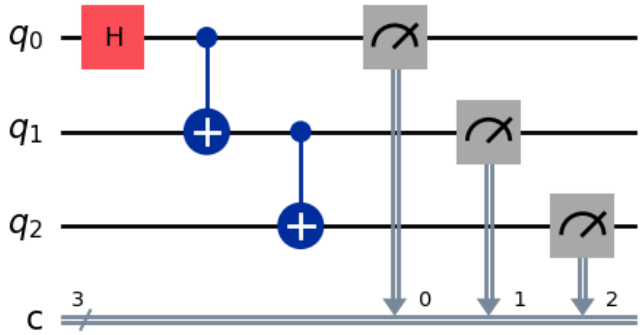


Figura 7: Diagrama do circuito para o estado GHZ, ilustrando a propagação do emaranhamento em cascata.

condicional de operações, o que atua como um análogo quântico das estruturas de controle clássicas (*if-statements*). Na biblioteca Qiskit, isso é implementado através da abstração de circuitos em portas lógicas personalizadas e da utilização de métodos como `control` e `compose` (ou `append`).

Para demonstrar esse princípio, desenvolvemos um experimento onde a preparação de um estado de Bell é condicionada ao estado de um terceiro qubit de controle. O circuito base de Bell é transformado em uma operação única controlada.

Um aspecto fundamental deste experimento ocorre quando o qubit de controle é previamente colocado em superposição por uma porta Hadamard. O estado inicial do sistema de 3 qubits é $|000\rangle$. Ao aplicar a porta H no qubit de controle (q_0), obtemos:

$$|\psi_1\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |00\rangle = \frac{1}{\sqrt{2}}(|000\rangle + |100\rangle) \tag{19}$$

Quando a sub-rotina de Bell controlada é aplicada, ela age como a identidade caso $q_0 = |0\rangle$, e prepara o estado $|\Phi^+\rangle$ nos alvos (q_1 e q_2) caso $q_0 = |1\rangle$. Pelo princípio da linearidade [NC10], o estado final resultante é:

$$|\psi_{final}\rangle = \frac{1}{\sqrt{2}}|000\rangle + \frac{1}{2}|100\rangle + \frac{1}{2}|111\rangle \tag{20}$$

O código a seguir implementa essa arquitetura modular:

```
# 1. Sub-rotina independente (preparação de Bell)
subrotina_bell = QuantumCircuit(2, name='Prepara_Bell')
subrotina_bell.h(0)
subrotina_bell.cx(0, 1)

# 2. Criação da porta controlada (o "if" quântico)
porta_bell_controlada = subrotina_bell.to_gate().control
    (1)

# 3. Construção do circuito principal
circuito = QuantumCircuit(3, 3)
circuito.h(0) # Qubit de controle em superposição

# 4. Aplicação condicional da sub-rotina
circuito.append(porta_bell_controlada, [0, 1, 2])

# 5. Decomposição para simulação e medição
circuito_simulado = circuito.decompose()
circuito_simulado.measure([0, 1, 2], [0, 1, 2])
```

Código 8: Implementação de sub-rotina controlada condicionando operações através dos métodos `control` e `append`.

A simulação estatística deste circuito confirma a teoria: as medições retornam o estado $|000\rangle$ em aproximadamente 50% das execuções (quando o controle colapsa para 0), e os estados $|100\rangle$ e $|111\rangle$ em cerca de 25% cada (quando o controle colapsa para 1, ativando a rotina de Bell).

Também é possível utilizar portas que dependam de mais do que um qbit para seu controle. Um exemplo importante é a porta de Toffoli (figura 9), em que dois qbits de controle são usados para condicionar uma porta NOT.

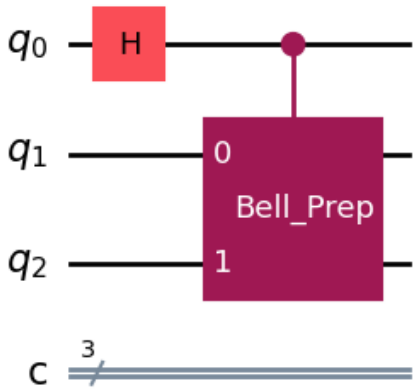


Figura 8: Abstração visual do circuito condicional, demonstrando a aplicação de uma sub-rotina modular controlada por um qubit em superposição.

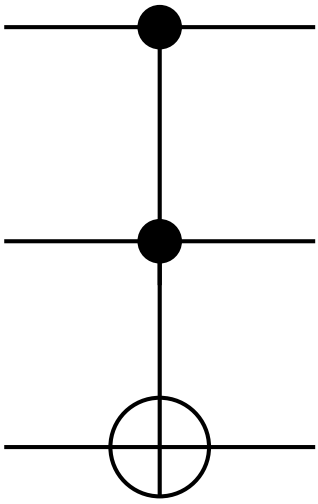


Figura 9: A porta de Toffoli. Se os qbits q_0 e q_1 estiverem no estado $|11\rangle$ uma porta NOT é aplicada em q_2.

4 Algoritmo de Grover

O algoritmo de Grover, proposto por Lov Grover em 1996 [Gro96, NC10], é o principal algoritmo quântico para busca em dados não estruturados. Inicia-se com uma superposição de todas as possibilidades. Grover aumenta a probabilidade (amplitude) da resposta correta e diminui a probabilidade das respostas erradas. Isso é feito através de um processo geométrico. Sejam dois vetores:

1. $|\omega\rangle$: o estado alvo que queremos encontrar.
2. $|s\rangle$: o estado de superposição uniforme (média de todos os estados).

O algoritmo gira o vetor de estado atual em direção a $|\omega\rangle$. Segue o processo geométrico:

1. Começamos no estado de superposição $|s\rangle$.
2. O oráculo reflete o vetor, invertendo a fase do item vencedor.
3. O difusor reflete o vetor em torno da média.
4. O resultado é que o vetor se aproxima progressivamente do eixo vertical ($|\omega\rangle$).

Para implementar esse algoritmo, precisa-se de três componentes principais:

A. Inicialização (superposição): começamos com todos os qubits em estado zero $|00...0\rangle$ e aplicamos a porta H a todos. Isso cria uma superposição onde todas as N respostas possíveis têm a mesma chance de serem medidas.

B. O oráculo (U_f): é a "caixa-preta". É uma função que reconhece a resposta correta. Se o estado não é correto ($x \neq \omega$), ele não faz nada. Se o estado é o correto ($x = \omega$), ele inverte o sinal (fase) desse estado: $U_f|x\rangle = (-1)^{f(x)}|x\rangle$, onde $f(x) = 1$ para a resposta correta e $f(x) = 0$ caso contrário. Dessa forma, $|\omega\rangle \longrightarrow -|\omega\rangle$. Isso "marca" a resposta

correta matematicamente, mas se o sistema fosse medido imediatamente, a probabilidade seria a mesma (pois a probabilidade é o quadrado da amplitude e $|-a|^2 = |a|^2$). Precisa-se, pois, do próximo passo para tornar essa marcação detectável.

C. O difusor (operador Grover): realiza a inversão sobre a média. Matematicamente, ele toma todas as amplitudes e as espelha em relação à média das amplitudes. Como o oráculo deixou a amplitude do $|\omega\rangle$ negativa (muito abaixo da média positiva), ao inverter sobre a média, a amplitude do $|\omega\rangle$ dispara para cima (tornando-se fortemente positiva), enquanto as outras diminuem.

Por exemplo, para determinar se um grafo possui um ciclo hamiltoniano, os métodos clássicos tradicionais de busca exaustiva (como retrocesso ou busca em profundidade) exigem tempo exponencial. No contexto quântico, o algoritmo de Grover pode ser empregado para pesquisar sobre o espaço de permutações de vértices, utilizando um oráculo capaz de verificar eficientemente se a sequência proposta constitui um caminho válido.

4.1 O Oráculo Quântico (U_ω)

O primeiro componente fundamental do algoritmo de Grover é o oráculo, denotado por U_ω. Sua função principal é atuar como uma "caixa-preta" que identifica a solução do problema de busca sem revelar qual é. Para um espaço de busca $U = \{0, ..., 2^n - 1\}$ com n qubits, definimos o estado alvo como $|\omega\rangle$.

O oráculo aplica uma transformação condicional no estado do sistema. Se o estado avaliado não for a solução ($x \neq \omega$), o oráculo atua como a matriz identidade, deixando o estado inalterado. Se o estado for a solução ($x = \omega$), o oráculo inverte a sua fase (sinal). Matematicamente, isso é expresso como:

$$U_\omega|x\rangle = \begin{cases} |x\rangle & \text{se } x \neq \omega \\ -|x\rangle & \text{se } x = \omega \end{cases} \tag{21}$$

Na prática da construção de circuitos quânticos, a inversão de fase é frequentemente implementada através de uma porta Z multi-controlada

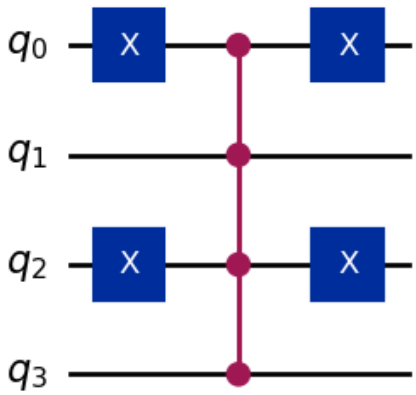


Figura 10: Estrutura interna do Oráculo U_5. As portas Pauli-X atuam como condicionais para ativar a porta Z multi-controlada (MCZ) apenas no estado $|0101\rangle$.

(MCZ). A porta Z aplica o fator -1 ao estado $|1\rangle$. Para que ela marque um estado específico arbitrário, utiliza-se o artifício de aplicar portas Pauli-X (NOT) antes e depois da operação controlada nos qubits correspondentes aos bits 0 do estado alvo.

Por exemplo, considere um sistema de $n = 4$ qubits onde desejamos encontrar o estado $\omega = 5$, cuja representação binária é $|0101\rangle$. A porta MCZ natural dispararia apenas para o estado $|1111\rangle$. Portanto, aplicamos portas X no primeiro e no terceiro qubit (onde temos 0 na string alvo). Isso converte temporariamente o estado $|0101\rangle$ em $|1111\rangle$, acionando a MCZ, que inverte a fase global do termo. Em seguida, aplicamos as portas X novamente para restaurar o estado original, que agora carrega a fase negativa: $-|0101\rangle$.

4.2 Gerenciamento de Ancillas e Descomputação

No design e na análise de algoritmos quânticos, o gerenciamento de recursos (qubits) é tão crítico quanto a alocação de memória na computação clássica. Ao implementar portas multi-controladas complexas para oráculos grandes, as limitações topológicas do hardware exigem a decomposição de operações em

portas menores (como portas Toffoli). Esse processo frequentemente requer a utilização de qubits auxiliares, conhecidos como *ancillas*, para armazenar resultados intermediários de controle.

O uso de ancillas impõe uma restrição fundamental: a necessidade de **descomputação** (*uncomputing*). Após o resultado intermediário ter sido utilizado para acionar a operação no qubit alvo, as operações aplicadas nas ancillas devem ser revertidas espelhando o circuito. Se as ancillas não forem retornadas perfeitamente ao seu estado inicial (geralmente $|0\rangle$), elas permanecerão emaranhadas com o registro de processamento principal. Esse fenômeno gera lixo computacional (*garbage entanglement* [NC10]), que destrói a coerência do sistema e impede que a interferência construtiva do difusor amplifique a probabilidade do estado correto.

4.3 O Difusor e a reflexão de Householder

Após a marcação pelo oráculo, o algoritmo aplica o operador de difusão de Grover, denotado por U_s. Enquanto o oráculo marca a solução alterando sua fase geométrica, o difusor redistribui as amplitudes de probabilidade do sistema, realizando a chamada "inversão sobre a média".

A estrutura matemática do difusor é intrinsecamente ligada ao conceito de **Reflexão de Householder** da álgebra linear. Uma transformação de Householder reflete um vetor no espaço em relação a um plano ortogonal a um dado vetor normal. O difusor U_s pode ser interpretado como uma reflexão do vetor de estado do sistema em torno do estado de superposição uniforme $|s\rangle$. É construído a partir do operador de inversão ao redor do estado zero $U_0 = \mathbb{I} - 2|0\rangle\langle 0|$, conjugado por portas Hadamard:

$$U_s = H^{\otimes n}(\mathbb{I} - 2|0\rangle\langle 0|)H^{\otimes n} = 2|s\rangle\langle s| - \mathbb{I} \tag{22}$$

A operação $U_0|x\rangle$ atua preservando o estado se $x \neq 0$ e invertendo seu sinal se $x = 0$ ($-|0\rangle$).

A essência geométrica do algoritmo de Grover reside na combinação destas duas reflexões sequenciais: a primeira pelo oráculo (U_ω) e a segunda pelo difusor (U_s). O produto de duas reflexões no plano bidimensional gerado por

$|\omega\rangle$ e o estado ortogonal a ele resulta em uma **rotação pura**. A cada iteração de Grover, o vetor de estado do sistema é rotacionado em direção ao estado alvo $|\omega\rangle$, amplificando progressivamente sua probabilidade de medição com uma aceleração quadrática em relação a qualquer método de busca exaustiva clássico.

4.4 Implementação do Algoritmo de Grover

A implementação prática do algoritmo de Grover foi estruturada de forma modular. O Oráculo e o Difusor foram desenvolvidos como blocos independentes e posteriormente acoplados através do método `append()` no circuito principal, facilitando a reutilização dos componentes.

```
# 1. Construção do Oráculo (U_w) para o estado |0101>
oraculo = QuantumCircuit(4, name="Oraculo (Marca 5)")
oraculo.x([0, 2]) # Inverte os bits 0 do estado alvo
porta_mcz = ZGate().control(3) # Z multi-controlada
oraculo.append(porta_mcz, [0, 1, 2, 3])
oraculo.x([0, 2]) # Descomputação

# 2. Construção do Difusor (U_s) - Inversão sobre a mé
   dia
difusor = QuantumCircuit(4, name="Difusor (Inversão)")
difusor.h([0, 1, 2, 3])
difusor.x([0, 1, 2, 3])
difusor.append(porta_mcz, [0, 1, 2, 3])
difusor.x([0, 1, 2, 3])
difusor.h([0, 1, 2, 3])

# 3. Montagem do circuito principal de Grover
circuito_grover = QuantumCircuit(4, 4)
circuito_grover.h([0, 1, 2, 3]) # Superposição uniforme

# Aplicação iterativa do Oráculo e Difusor (3 iterações)
for rodada in range(3):
```

```
22      circuito_grover.append(oraculo.to_instruction(), [0,
        1, 2, 3])
23      circuito_grover.append(difusor.to_instruction(), [0,
        1, 2, 3])
24
25  circuito_grover.measure([0, 1, 2, 3], [0, 1, 2, 3])
```

Código 9: Implementação dos componentes do Algoritmo de Grover: Oráculo para o estado alvo $|0101\rangle$ e Difusor de Householder.

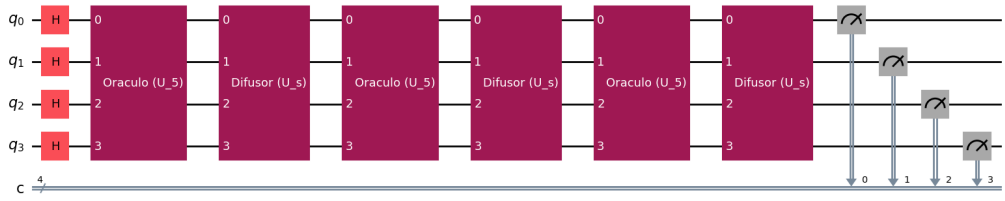


Figura 11: Circuito principal do Algoritmo de Grover abstraído em blocos modulares. O processo evidencia a inicialização em superposição seguida de 3 iterações consecutivas do Oráculo e Difusor.

4.5 Complexidade do algoritmo de Grover

A superioridade do algoritmo de Grover reside na sua aceleração quadrática em relação a qualquer algoritmo clássico de busca não estruturada. Classicamente, para encontrar um item específico em uma lista não ordenada de N elementos, o pior cenário exige a verificação de todos os itens, resultando em uma complexidade de tempo $\mathcal{O}(N)$. Em média, são necessárias $N/2$ consultas ao banco de dados.

Quanticamente, a complexidade é analisada pelo número de vezes que o oráculo U_ω precisa ser invocado. Geometricamente, o estado inicial de superposição uniforme $|s\rangle$ forma um pequeno ângulo $\theta/2 \approx 1/\sqrt{N}$ com o plano ortogonal ao estado alvo $|\omega\rangle$. Como cada iteração de Grover (Oráculo + Difusor) rotaciona o vetor de estado por um ângulo fixo de $\theta \approx 2/\sqrt{N}$ em direção a $|\omega\rangle$, o número de iterações necessárias para que o vetor atinja

o eixo de $|\omega\rangle$ (uma rotação total de $\approx \pi/2$) é assintoticamente dado por $R \approx \frac{\pi/2}{\theta} \approx \frac{\pi}{4}\sqrt{N}$ [Gro96, NC10].

Portanto, a complexidade temporal do algoritmo de Grover é $\mathcal{O}(\sqrt{N})$ [NC10]. Para um espaço de busca de 1 milhão de itens ($N = 10^6$), enquanto um computador clássico faria cerca de 500.000 consultas, um computador quântico encontraria a resposta com certeza probabilística em apenas ≈ 785 consultas.

Também é possível estender o algoritmo para situações onde existem K alvos (ou seja tal que $f(x) = 1$ para K valores distintos). O oráculo nessa variante continua sendo tal que $U_f|x\rangle = (-1)^{f(x)}|x\rangle$ e a única modificação é que como o ângulo de rotação muda, precisamos fazer por volta de $\frac{\pi}{4}\sqrt{\frac{N}{K}}$ aplicações do par oráculo-difusor.

5 O Algoritmo BHT e o Problema da Colisão

5.1 O Problema da Colisão em Funções 2-para-1

Proposto por Gilles Brassard, Peter Høyer e Alain Tapp em 1997 [BHT97, NC10], o algoritmo BHT resolve o *Problema da Colisão* (*Collision Problem*). Dada uma função $f : \{0, \ldots, N-1\} \to \{0, \ldots, N/2-1\}$ que é 2-para-1, ou seja, onde cada imagem y possui exatamente duas pré-imagens distintas $x_1 \neq x_2$ tais que $f(x_1) = f(x_2)$, o objetivo consiste em identificar um par de colisão (x_1, x_2). Esse problema fundamenta a análise de segurança de funções de *hash* criptográficas e esquemas de assinatura digital. Classicamente, a melhor abordagem probabilística, baseada no Paradoxo do Aniversário, requer $\mathcal{O}(N^{1/2})$ consultas à função.

5.2 A Estratégia Híbrida e a Construção do Oráculo BHT

O algoritmo BHT supera a barreira clássica do paradoxo do aniversário ao combinar amostragem clássica com a busca de Grover em um protocolo híbrido de duas etapas:

1. **Amostragem Clássica:** Sorteia-se um subconjunto de $M = \lceil N^{1/3} \rceil$ entradas aleatórias distintas x_k e armazenam-se os pares $(x_k, f(x_k))$ em uma tabela *hash*. Se houver colisão na própria amostra, o processo é concluído classicamente.

2. **Busca Quântica de Grover:** Se não houver colisão na tabela, existem exatamente M entradas fora da amostra que colidem com os valores previamente tabelados. Aplica-se a busca de Grover sobre o espaço de dimensão N para localizar um estado x^* cuja imagem $f(x^*)$ coincida com algum valor da tabela clássica.

Para implementar o oráculo quântico V que reconhece as colisões, decompõe-se o operador unitário na estrutura modular:

$$V = F^\dagger U F \tag{23}$$

Aqui F é um operador unitário, que "implementa" a função f, de forma que $F|x\rangle = |\phi(x)\rangle$, onde ϕ é uma permutação de $\{0, \ldots, N-1\}$ e $f(x) = \lfloor \phi(x)/2 \rfloor$. Já U é um operador diagonal de inversão de fase atuando sobre as imagens:

$$U|y\rangle \otimes |s\rangle = \begin{cases} -|y\rangle \otimes |s\rangle, & \text{se } y \in \{f(x_k) \mid x_k \in S\} \\ |y\rangle \otimes |s\rangle, & \text{caso contrário.} \end{cases} \tag{24}$$

onde $s = 0$ ou 1.

Dessa forma, a aplicação de F codifica a imagem no estado quântico, U inverte a fase das saídas coincidentes com a tabela clássica, e $F^\dagger$ atua como a descomputação quântica necessária para retornar à base dos índices, garantindo que o oráculo V aja como $V|x\rangle = -|x\rangle$ precisamente nos estados x que geram colisões.

5.3 Implementação do Algoritmo BHT

A implementação prática do algoritmo concentra-se na construção do oráculo modular $V = F^\dagger U F$ acoplado ao difusor de Householder. Nesse exemplo, usamos um domínio de 4 qubits ($N = 16$ itens):

```
# 1. Função 2-para-1 (4 Qubits -> N = 16 elementos)
n_qubits = 4
N = 2 ** n_qubits
np.random.seed(42)
phi = np.random.permutation(N)
f = phi // 2  # Cada valor de 0 a 7 aparece 2 vezes

# 2. Amostragem classica de M = ceil(N^(1/3)) elementos
M = int(np.ceil(N ** (1 / 3)))
amostras_x = list(np.random.choice(N, size=M, replace=
    False))
tabela = {x: f[x] for x in amostras_x}
alvos = set(tabela.values())

# 3. Operador Unitario do Oraculo V = F_dag @ U @ F
matriz_F = np.eye(N)[phi].T # Permutacao
matriz_U = np.diag([-1.0 if (y // 2) in alvos else 1.0
    for y in range(N)])
matriz_V = matriz_F.T @ matriz_U @ matriz_F

# 4. Circuito Quantico de Grover (Oraculo V + Difusor D)
circuito_bht = QuantumCircuit(n_qubits, n_qubits)
circuito_bht.h(range(n_qubits)) # Superposicao uniforme
circuito_bht.append(UnitaryGate(matriz_V, label="Oraculo
    V"), range(n_qubits))
circuito_bht.append(UnitaryGate(matriz_D, label="Difusor
    D"), range(n_qubits))
circuito_bht.measure(range(n_qubits), range(n_qubits))

# 5. Recuperacao do par de colisao (x1, x2)
x_quantico = int(max(contagens, key=contagens.get), 2)
```

```
x_classico = [k for k, v in tabela.items() if v == f[
    x_quantico]][0]
```

Código 10: Motor do Algoritmo BHT: Construção do oráculo $V = F^{\dagger}UF$ e busca de colisão no Qiskit.

O circuito quântico completo montado no Qiskit é ilustrado na Figura 12, e o histograma com a distribuição de probabilidades resultante da simulação no `AerSimulator` é apresentado na Figura 13, comprovando a amplificação seletiva das amplitudes dos estados associados à colisão.

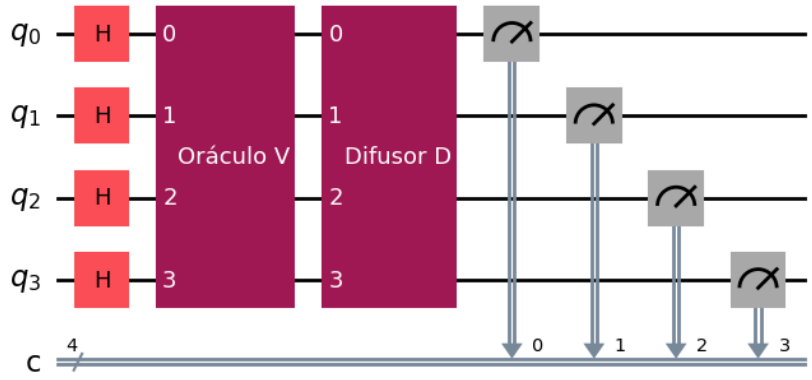


Figura 12: Circuito quântico do Algoritmo BHT estruturado com o operador de superposição $H^{\otimes 4}$, o oráculo unitário $V = F^{\dagger}UF$ e o difusor de Householder D, seguido pela medição na base computacional.

5.4 Complexidade do Algoritmo BHT

A complexidade temporal do algoritmo BHT é avaliada pelo número total de consultas (*queries*) à caixa-preta f:

- A etapa de amostragem clássica consome $M = N^{1/3}$ consultas à função.
- Como existem $M = N^{1/3}$ alvos em um espaço amostral de tamanho N, o número ótimo de iterações de Grover é:

$$R \approx \frac{\pi}{4}\sqrt{\frac{N}{M}} = \frac{\pi}{4}\sqrt{\frac{N}{N^{1/3}}} = \frac{\pi}{4}N^{(1-1/3)/2} = \mathcal{O}(N^{1/3}) \tag{25}$$

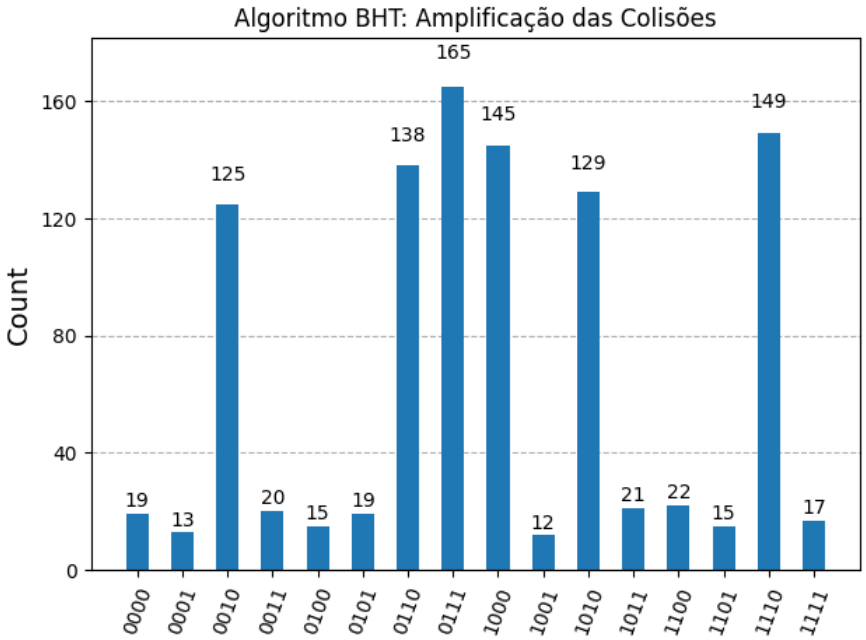


Figura 13: Distribuição de probabilidades obtida no simulador após uma iteração de Grover, evidenciando a amplificação de amplitude dos estados associados às colisões. Como $M = 3$ no caso $N = 16$, devemos ter exatamente 6 estados com a probabilidade amplificada.

Somando as duas fases, o custo computacional total do algoritmo BHT é $\mathcal{O}(N^{1/3}) + \mathcal{O}(N^{1/3}) = \mathcal{O}(N^{1/3})$ consultas, superando a barreira clássica do Paradoxo do Aniversário de $\mathcal{O}(N^{1/2})$ e atingindo o limite inferior ótimo para o problema da colisão em caixas-pretas [NC10].

6 Transformada de Fourier Quântica (QFT)

A Transformada de Fourier Quântica (QFT) é a contraparte quântica da Transformada Discreta de Fourier (DFT). No domínio clássico, a DFT age como um "prisma matemático", decompondo um sinal (um vetor de N números complexos) em suas frequências constituintes, gerando um novo vetor de N dados acessíveis.

No domínio quântico, a QFT realiza a mesma transformação algébrica, mas opera estritamente sobre as **amplitudes de probabilidade** de um

estado no Espaço de Hilbert, mudando a codificação da base computacional para a base de Fourier. A definição matemática para um estado da base $|x\rangle$ é:

$$QFT|x\rangle = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} e^{\frac{2\pi i x y}{N}} |y\rangle \tag{26}$$

6.1 A Falsa Equivalência e o Paradoxo da Medição

É imperativo distinguir a utilidade da QFT em relação à Transformada Rápida de Fourier clássica (FFT). Enquanto a FFT clássica exige $O(N \log N)$ operações, a QFT processa os mesmos $N = 2^n$ dados usando apenas $O(n^2)$ portas lógicas. Para um sistema de 100 qubits ($N = 2^{100}$), a aceleração é massivamente exponencial.

Contudo, **a QFT não pode ser utilizada como um substituto direto para processar sinais clássicos**. Na computação clássica, o vetor de saída contendo todas as frequências pode ser lido inteiramente. Na computação quântica, o resultado da QFT é um estado em superposição. Pelos postulados da mecânica quântica, a medição causa o colapso irreversível da função de onda. Ao tentar "ler" o resultado da QFT, o observador obtém apenas um único estado base $|y\rangle$ com probabilidade $|c_y|^2$, destruindo imediatamente todas as outras frequências calculadas.

Portanto, a QFT demonstra seu poder não no processamento de dados brutos, mas na extração de **propriedades globais** escondidas em um sistema quântico (como a periodicidade). Ela é projetada para causar padrões de interferência onde a resposta correta (a frequência dominante do período) concentra quase toda a amplitude de probabilidade, permitindo que uma única medição revele o segredo estrutural da função analisada.

6.2 Implementação e a Representação em Produto

A eficiência da QFT deriva da constatação de que a somatória de Fourier pode ser fatorada analiticamente em um produto tensorial de estados independentes

para cada qubit:

$$QFT|x\rangle = \bigotimes_{k=1}^{n} \left(\frac{|0\rangle + e^{\frac{2\pi i x}{2^k}}|1\rangle}{\sqrt{2}} \right) \tag{27}$$

O circuito requer apenas duas ferramentas:

1. **Portas Hadamard (H):** Criar a superposição inicial.
2. **Portas de Fase Controlada (R_k):** Aplicam rotações fracionárias da fase relativa. A matriz de rotação é $R_k = \text{diag}(1, e^{2\pi i/2^k})$.
3. **Portas SWAP:** Corrigem o *endianness* do resultado.

A topologia geral do circuito da QFT para múltiplos qubits, incorporando a cascata de portas de fase controlada e as portas unitárias `SWAP` finais, é ilustrada na Figura 14 (com 4 qbits).

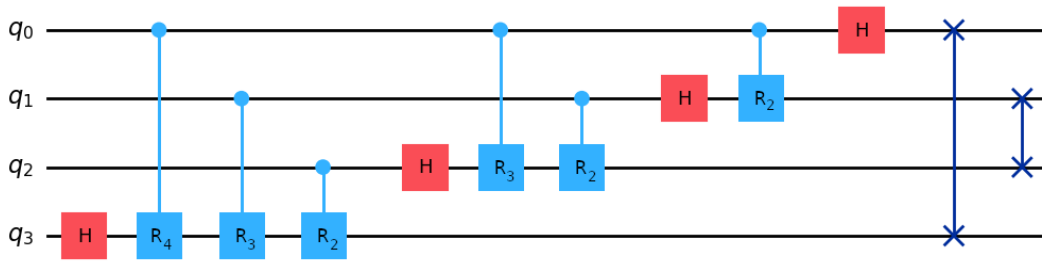


Figura 14: Estrutura do circuito quântico da Transformada de Fourier Quântica (QFT), destacando as portas Hadamard H, as rotações de fase controlada R_k e as portas SWAP finais para reversão da ordem dos qubits.

6.3 A Necessidade das Portas SWAP: Uma Prova Algébrica

A fatoração por produto tensorial resulta na codificação das fases em uma ordem de significância invertida (bit-reversal). Para demonstrar essa inversão, analisa-se um sistema de 2 qubits ($N = 4$) inicializado no estado $|x = 3\rangle$.

Ao aplicarmos a fórmula direta da somatória sobre $|x = 3\rangle$, o estado esperado é:

$$|\psi_{esp}\rangle = \frac{1}{2}\left(|0\rangle - i|1\rangle - |2\rangle + i|3\rangle\right) \tag{28}$$

No entanto, a compilação teórica das portas de rotação H e R_k sem correção resulta no seguinte estado de saída no circuito:

$$|\psi_{circ}\rangle = \frac{1}{2}\left(|00\rangle - |01\rangle - i|10\rangle + i|11\rangle\right) \tag{29}$$

Observa-se uma anomalia geométrica: a amplitude $-i$, que pertence ao estado $|1\rangle$ (binário 01), foi alocada no estado $|10\rangle$ (decimal 2). Para alinhar o *output* físico à convenção matemática clássica, é mandatório aplicar portas unitárias `SWAP` ao final do circuito para inverter a ordem espacial dos qubits.

6.4 Complexidade do Algoritmo da Transformada de Fourier Quântica (QFT)

A eficiência da QFT é o motor por trás da aceleração exponencial em algoritmos de busca de período. Para processar um vetor de estado de $N = 2^n$ amplitudes (onde n é o número de qubits), o melhor algoritmo clássico, a Transformada Rápida de Fourier (FFT), requer $\mathcal{O}(N \log N)$ operações. Substituindo N, a complexidade clássica escala como $\mathcal{O}(n2^n)$, um crescimento exponencial em função do número de bits.

No circuito quântico, a construção da QFT é avaliada pela contagem de portas lógicas elementares. Para o primeiro qubit, aplica-se uma porta Hadamard e $n-1$ portas de rotação controlada (R_k). Para o segundo, uma porta Hadamard e $n-2$ rotações, e assim sucessivamente até o último qubit, que recebe apenas uma Hadamard.

Essa progressão aritmética resulta em uma contagem exata de portas dada por $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$. Adicionando as $\mathcal{O}(n)$ portas SWAP necessárias no final do circuito, a complexidade total do algoritmo assintoticamente converge para $\mathcal{O}(n^2)$ [NC10]. Isso representa uma extraordinária aceleração exponencial, executando uma transformação global sobre 2^n estados utilizando um número de recursos que cresce apenas polinomialmente.

7 Algoritmo de Estimação de Fases (QPE)

A Estimação de Fases Quântica (QPE - *Quantum Phase Estimation*), introduzida originalmente por Alexei Kitaev [Kit95], é uma das sub-rotinas mais cruciais da computação quântica. Ela atua como a ponte entre a manipulação de fases abstratas e a extração de dados clássicos mensuráveis.

O problema fundamental que a QPE resolve é o seguinte: dado um operador unitário U e um estado quântico $|\Psi\rangle$ que é autovetor desse operador, desejamos encontrar o autovalor associado. Como U é unitário e preserva a conservação da probabilidade, o seu autovalor deve obrigatoriamente ter módulo 1. Pela fórmula de Euler, podemos escrever esse autovalor estritamente como uma fase complexa:

$$U|\Psi\rangle = e^{2\pi i\theta}|\Psi\rangle \tag{30}$$

onde θ é um número real desconhecido ($0 \leq \theta < 1$) que desejamos estimar com alta precisão.

7.1 A Mecânica do Phase Kickback

Para extrair o valor de θ sem destruir o estado $|\Psi\rangle$ através de uma medição ineficaz, o algoritmo utiliza o artifício geométrico do *Phase Kickback*. O circuito é dividido em dois registradores principais:

1. **Registrador Alvo (Target):** Inicializado com o autovetor $|\Psi\rangle$. Ele sofrerá a ação das portas unitárias, mas, por ser um autovetor, permanecerá intacto do ponto de vista da medição.

2. **Registrador de Controle (Leitura):** Composto por n qubits inicializados em $|0\rangle$, que são colocados em superposição uniforme através de portas Hadamard.

Ao aplicarmos o operador U condicionado a um qubit do registrador de controle, a fase $e^{2\pi i\theta}$ é "chutada"de volta (kickback) para o qubit de controle. O estado do qubit alvo não muda, mas a amplitude do qubit de controle na

base $|1\rangle$ absorve a fase relativa do autovalor. Algebricamente, o operador que age em um qbit de controle q decidindo se U será aplicado em $|\psi\rangle$ é

$$|0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U \tag{31}$$

de forma que se esse operador age em $(H|0\rangle) \otimes |\psi\rangle$ temos

$$(|0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U) \left(\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) \otimes |\psi\rangle \right) =$$

$$= \frac{1}{\sqrt{2}}(|0\rangle \otimes |\psi\rangle + |1\rangle \otimes (U|\psi\rangle)) = \left(\frac{|0\rangle + e^{2\pi i\theta}|1\rangle}{\sqrt{2}} \right) \otimes |\psi\rangle \tag{32}$$

ou seja, q absorve a fase relativa.

7.2 Codificação em Potências de U

Para obter a leitura exata de θ em formato binário fracionário, um único evento de *kickback* é insuficiente. O algoritmo QPE aplica o operador U em potências sucessivas de base 2 ($U^1, U^2, U^4, \ldots, U^{2^{n-1}}$), condicionando cada potência a um dos n qubits distintos do registrador de controle.

Ao elevar o operador a uma potência k, o autovalor associado é matematicamente elevado à mesma potência: $U^k|\Psi\rangle = (e^{2\pi i\theta})^k|\Psi\rangle = e^{2\pi ik\theta}|\Psi\rangle$. Esse processo "fatia" a fase θ, permitindo que cada qubit de controle reaja a uma casa binária específica do ângulo. Após essa cascata de operações condicionais, o estado isolado do registrador de controle (onde $N = 2^n$) torna-se:

$$|\Psi_{controle}\rangle = \frac{1}{\sqrt{N}} \sum_{s=0}^{N-1} e^{2\pi is\theta}|s\rangle \tag{33}$$

7.3 A Transformada de Fourier Inversa como Decodificador

O estado resultante no registrador de controle contém a informação do ângulo θ perfeitamente codificada e espalhada em suas fases complexas. Nota-se que este estado é algebricamente idêntico à saída padrão de uma Transformada de Fourier Quântica.

Para converter essas frequências ocultas de volta em um número inteiro legível na base computacional, aplica-se a Transformada de Fourier Quântica Inversa (QFT^{-1}) sobre o registrador de controle. A QFT^{-1} atua como um decodificador de fases, induzindo um padrão de interferência onde as amplitudes erradas se cancelam (interferência destrutiva) e a probabilidade correta se concentra no estado $|m\rangle$, tal que $m \approx N\theta$.

Ao final do circuito, uma simples medição na base Z do registrador de controle forçará o colapso para o inteiro m. Conhecendo o valor de m e a dimensão N do registrador, recupera-se o autovalor original através da divisão empírica $\theta = m/N$.

7.4 Implementação do Algoritmo de Estimação de Fases (QPE)

Para validar empiricamente o formalismo do *Phase Kickback* e da QFT^{-1}, implementamos o algoritmo de Estimação de Fases (QPE) utilizando o *framework* Qiskit. O objetivo do experimento foi descobrir uma fase geométrica fracionária, injetada artificialmente em um sistema, utilizando a interferência quântica.

O operador unitário escolhido para o teste foi a porta de fase $P(\lambda)$. A propriedade fundamental deste operador é que ele atua como a matriz identidade para o estado $|0\rangle$. Portanto, sabemos *a priori* que $|1\rangle$ é o autovetor do nosso operador. Definimos a fase secreta geométrica como $\theta = 1/8$ (0.125), o que corresponde a um ângulo real de rotação $\lambda = 2\pi(1/8) = \pi/4$.

Para o registrador de controle (leitura), estipulamos $t = 3$ qubits, estabelecendo uma dimensão de espaço de $N = 2^3 = 8$. Esta configuração garante que o circuito possua precisão suficiente para medir fases fracionárias em oitavos exatos.

```
from qiskit.circuit.library import QFT

# 1. Preparação dos registradores (3 leitura + 1 alvo)
qubits_leitura = 3
circuito_qpe = QuantumCircuit(qubits_leitura + 1,
```

```
    qubits_leitura)
circuito_qpe.h(range(qubits_leitura)) # Superposição na
    leitura
circuito_qpe.x(qubits_leitura)          # Autovetor |1>
    no alvo

# 2. Phase Kickback com potências de 2 (U^1, U^2, U^4)
angulo_rotacao = 2 * np.pi * (1 / 8) # Fase theta = 1/8
repeticoes = 1
for q_controle in range(qubits_leitura):
    for _ in range(repeticoes):
        circuito_qpe.cp(angulo_rotacao, q_controle,
    qubits_leitura)
    repeticoes *= 2 # Escalonamento exponencial

# 3. Decodificador: QFT Inversa
decodificador_qft = QFT(num_qubits=qubits_leitura,
    inverse=True, do_swaps=True).to_gate()
circuito_qpe.append(decodificador_qft, range(
    qubits_leitura))
circuito_qpe.measure(range(qubits_leitura), range(
    qubits_leitura))
```

Código 11: Implementação do algoritmo de Estimação de Fases (QPE) para extração da fase $\theta = 1/8$.

O circuito gerado, demonstrando a aplicação escalonada das portas unitárias controladas e a subsequente decodificação via Transformada de Fourier Inversa, pode ser visualizado na Figura 15.

Ao executar o circuito no `AerSimulator`, o histograma resultante (Figura 16) confirmou a precisão algébrica do algoritmo. O sistema colapsou com 100% de probabilidade para o estado binário `001`.

Convertendo o vetor de estado medido para a base decimal, obtemos o inteiro $m = 1$. Ao aplicarmos a equação fundamental de resolução do algoritmo, recuperamos a fase com precisão absoluta: $\theta = m/N \implies \theta = 1/8 = 0.125$. Como θ é da forma $n/8$ com n inteiro o algoritmo se comporta

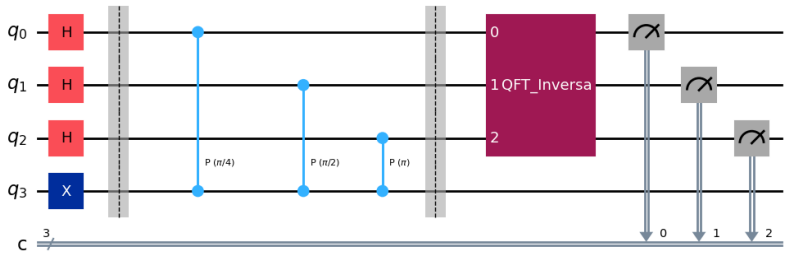


Figura 15: Circuito lógico da Estimação de Fases. Nota-se o crescimento exponencial nas aplicações das portas de rotação controlada (1, 2 e 4 operações consecutivas) atuando antes do bloco da QFT^{-1}.

de maneira determinística, porém qualquer θ que não seja uma fração diádica levaria a um comportamento estocástico, com a probabilidade de obter a melhor aproximação diádica como resposta aumentando com o número de qbits empregados (a qualidade dessa melhor aproximação também aumenta com o número de qbits).

7.5 Complexidade do Algoritmo de Estimação de Fases (QPE)

A complexidade da Estimação de Fases Quântica (QPE) é determinada pela soma dos recursos de suas duas etapas principais: a aplicação condicional das potências do operador unitário (U) e a Transformada de Fourier Quântica Inversa (QFT^{-1}).

Para estimar a fase geométrica com t bits de precisão, o registrador de controle é alocado com t qubits. A sub-rotina de decodificação (QFT^{-1}) exige, como demonstrado anteriormente, $\mathcal{O}(t^2)$ portas lógicas. Contudo, se não existir uma maneira mais simples de se obter U^n do que aplicar U n vezes, o gargalo computacional da QPE vai residir na fase de *Phase Kickback*. O operador U deve ser aplicado de forma controlada sucessivas vezes para alinhar as frequências, totalizando $2^0 + 2^1 + \cdots + 2^{t-1} = 2^t - 1$ aplicações [NC10].

Logo, a QPE requer $\mathcal{O}(t^2)$ portas lógicas de rotina geométrica e (nesse

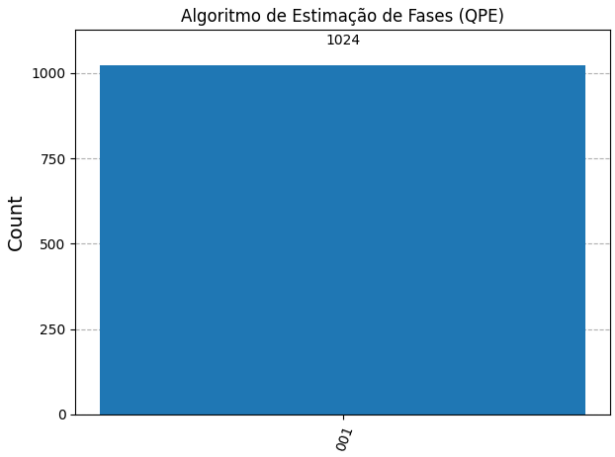


Figura 16: Histograma de resultados da QPE para $t = 3$ e $\theta = 0.125$. A interferência destrutiva suprime a probabilidade de colapso de todos os estados incorretos, isolando a resposta correta.

pior caso) $\mathcal{O}(2^t)$ invocações controladas do operador U. Logo a viabilidade de uso da QPE em algoritmos eficientes depende inteiramente da capacidade de otimizar ou pré-computar as potências de U^k de forma eficiente (em tempo polinomial). O Algoritmo de Shor que veremos a seguir é um exemplo onde isso é possível.

8 Algoritmo de Shor

O algoritmo proposto por Peter Shor em 1994 [Sho94, NC10] resolve eficientemente o problema de determinar a **ordem modular**: achar o menor inteiro positivo r tal que $a^r \equiv 1 \pmod{N}$. No fundo, trata-se de encontrar o período da função modular $f(x) = a^x \pmod{N}$.

1. O algoritmo de Shor prepara um estado quântico que "codifica" essa função periódica.

2. A QFT é aplicada a esse estado.

3. Ao medir o estado após a QFT, o resultado mais provável que se pode obter está diretamente relacionado ao período r que é procurado.

O algoritmo de Shor é famoso pela solução do problema da fatoração de números inteiros de maneira eficiente, porém o cerne do algoritmo não é a fatoração em si, mas a redução do problema de fatoração em um problema de busca de período, que a QFT consegue resolver de forma exponencialmente mais rápida que qualquer método clássico até então conhecido.

8.1 Implementação do Algoritmo de Shor

A implementação prática do algoritmo de Shor revela a sua natureza híbrida. Ele não é executado integralmente no hardware quântico; em vez disso, utiliza o processador quântico como um co-processador especializado para resolver o gargalo computacional do problema: a busca pelo período (função modular). O algoritmo é dividido em três etapas fundamentais.

A primeira etapa, puramente clássica, consiste na escolha de um palpite a primo com relação ao número N que se deseja fatorar. Para a nossa demonstração, escolhemos fatorar $N = 15$ utilizando o palpite $a = 7$. A função modular $f(x) = a^x \pmod N$ gera classicamente uma sequência cíclica, cujo comportamento periódico pode ser visualizado na Figura 17.

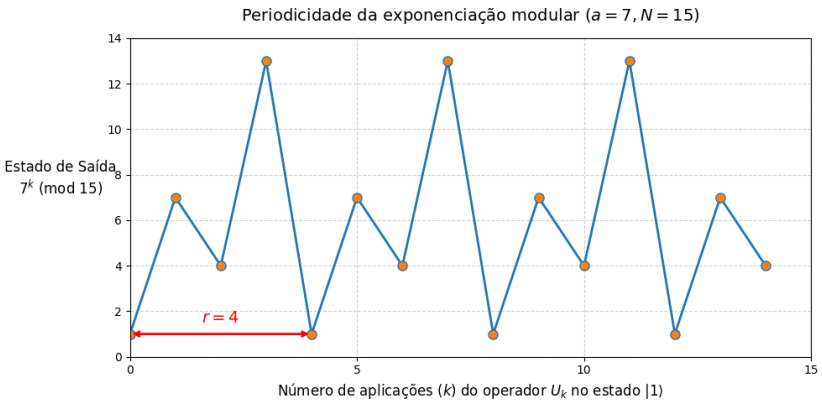


Figura 17: Comportamento periódico da função $f(k) = 7^k \pmod{15}$. Observa-se que a sequência de estados de saída se repete a cada 4 aplicações do operador, revelando o período $r = 4$.

A segunda etapa é o motor quântico. O problema de encontrar o período de f pode ser reformulado como o problema de se encontrar o período r da

unitária U tal que

$$U|k\rangle = \begin{cases} |ak \pmod N\rangle & , \text{ se } k < N \\ |k\rangle & , \text{ se } k \geq N \end{cases} \tag{34}$$

Se $\omega = e^{\frac{2\pi i}{r}}$, então por inspeção

$$|\psi_n\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} \omega^{kn} |a^k \pmod N\rangle \tag{35}$$

é um autovetor de U, com autovalor ω^{-n} para todo $n = 0, \ldots, r-1$. Também segue da eq (35) que

$$|1\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} |\psi_n\rangle \tag{36}$$

Logo, se eu aplicar um algoritmo de estimação de fases para a únitaria U mas usando $|1\rangle$ como a entrada no registrador alvo, o resultado vai ser igualmente distribuído entre os números da forma $\frac{n}{r}$ onde n é inteiro (isso segue da linearidade da transformação dada pelo circuito QPE como um todo mais o fato que eu sei os resultados se os alvos fossem os $|\psi_n\rangle$). Após rodarmos o algoritmo algumas vezes, teremos informações suficientes para deduzir r a partir dessas medições.

Para a implementação de U, precisamos implementar operações aritméticas, o que é possível de ser feito de maneira eficiente pois essas são apenas operações clássicas. Porém, a construção de um circuito aritmético quântico modular completo com portas Toffoli exige uma quantidade proibitiva de qubits para simuladores locais. Portanto, utilizamos uma abordagem otimizada: calculamos a matriz de permutação da função clássica e a encapsulamos em um operador unitário customizado através da classe `UnitaryGate` do Qiskit. Essa matriz é inserida diretamente na sub-rotina da Estimação de Fases Quântica (QPE) desenvolvida na seção anterior.

O código a seguir demonstra a união da matemática clássica com o circuito quântico de QPE para extrair o período escondido nas fases do operador unitário:

```
1 from qiskit.circuit.library import QFTGate
```

```
from fractions import Fraction

numero_N = 15
palpite_a = 7

# 1. Operador Unitário: |i> -> |(a*i) mod N>
def construir_U(a, N, total_qubits):
    dimensao = 2 ** total_qubits
    matriz_U = np.zeros((dimensao, dimensao))
    for i in range(dimensao):
        if i < N:
            matriz_U[(a * i) % N, i] = 1.0
        else:
            matriz_U[i, i] = 1.0
    return matriz_U

# 2. Circuito Quântico de Busca de Período (QPE)
qubits_contagem = 4
qubits_alvo = 4
circuito_shor = QuantumCircuit(qubits_contagem +
   qubits_alvo, qubits_contagem)

circuito_shor.h(range(qubits_contagem))
circuito_shor.x(qubits_contagem) # Inicializa alvo em
   |1>

matriz_base = construir_U(palpite_a, numero_N,
   qubits_alvo)
alvos = list(range(qubits_contagem, qubits_contagem +
   qubits_alvo))

for q_leitura in range(qubits_contagem):
    potencia = 2 ** q_leitura
    matriz_potencia = np.linalg.matrix_power(matriz_base
   , potencia)
```

```
32      porta_cu = UnitaryGate(matriz_potencia, label=f"U^{
    potencia}").control(1)
33      circuito_shor.append(porta_cu, [q_leitura] + alvos)
34
35  # 3. Decodificação via QFT Inversa e Medição
36  decodificador_qft = QFTGate(num_qubits=qubits_contagem).
    inverse()
37  circuito_shor.append(decodificador_qft, range(
    qubits_contagem))
38  circuito_shor.measure(range(qubits_contagem), range(
    qubits_contagem))
39
40  # 4. Pós-processamento Clássico (Frações Contínuas e MDC
    )
41  fase_medida = int(melhor_estado, 2) / (2 **
    qubits_contagem)
42  periodo_r = Fraction(fase_medida).limit_denominator(
    numero_N).denominator
43
44  if periodo_r % 2 == 0:
45      fator_1 = math.gcd(int(palpite_a ** (periodo_r / 2)
    - 1), numero_N)
46      fator_2 = math.gcd(int(palpite_a ** (periodo_r / 2)
    + 1), numero_N)
47      print(f"Fatores de {numero_N}: {fator_1} e {fator_2}
    ")
```

Código 12: Implementação híbrida do Algoritmo de Shor fatorando o número 15.

O circuito quântico modular montado no Qiskit para a fatoração de $N = 15$ é ilustrado na Figura 18, exibindo os quatro qubits de contagem, os quatro qubits de trabalho (alvo), os operadores unitários de exponenciação modular controlada e o bloco de decodificação via QFT^{-1}.

A terceira e última etapa ocorre após a medição quântica. O hardware retorna estados binários (como `0100` ou `1100`) que correspondem a frações

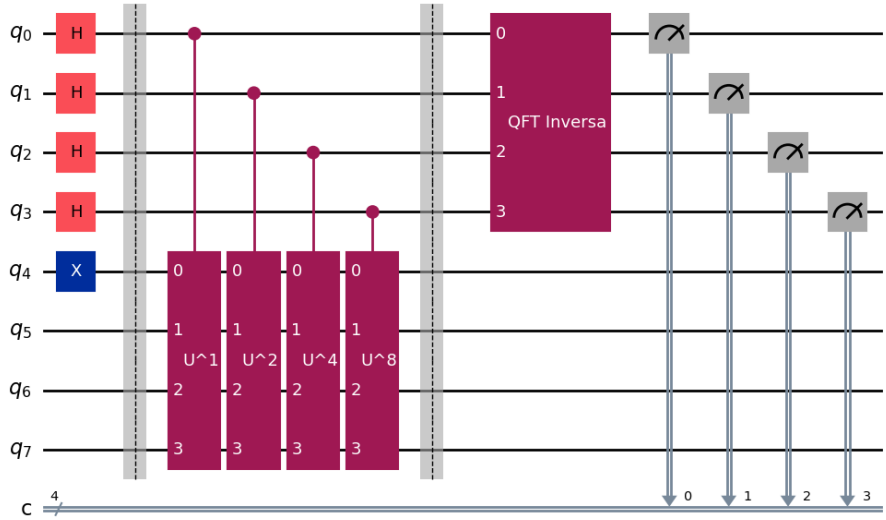


Figura 18: Circuito quântico para busca de período no Algoritmo de Shor ($N = 15$, $a = 7$), integrando a preparação em superposição, as portas modulares controladas U^{2^j} e a Transformada de Fourier Quântica Inversa (QFT^{-1}).

decimais da fase (por exemplo, 0,25 ou 0,75). Através do algoritmo clássico de Frações Contínuas (`limit_denominator`), o denominador da fração mais simples é extraído, revelando de forma determinística o período $r = 4$. Com esse período, sendo ele par, podemos usar o seguinte raciocínio:

$$a^{2s} - 1 = NM \Rightarrow (a^s - 1)(a^s + 1) = NM \tag{37}$$

Logo usando o algoritmo de Euclides, obtemos os candidatos $\text{mdc}(a^s - 1, N)$ e $\text{mdc}(a^s - 1, N)$ a fatores de N. No nosso caso ($N = 15$, $a = 7$ e $r = 4$) esses dão $\text{mdc}(48, 15) = 3$ e $\text{mdc}(50, 15) = 5$, quebrando com sucesso a criptografia do número 15.

8.2 Complexidade do Algoritmo de Shor

O algoritmo de Shor consolida a supremacia quântica ao resolver o problema de fatoração de inteiros em tempo polinomial. Classicamente, o algoritmo mais eficiente conhecido é o *General Number Field Sieve* (GNFS), que opera

em tempo super-polinomial: $\mathcal{O}\left(\exp\left(c(\log N)^{1/3}(\log\log N)^{2/3}\right)\right)$. Na prática, isso torna a fatoração de chaves criptográficas (como o RSA de 2048 bits) um problema intratável para a arquitetura clássica.

A abordagem de Shor converte a fatoração no problema de busca do período r da função modular $f(x) = a^x \pmod N$, utilizando a sub-rotina QPE. O processamento puramente clássico de pré e pós-processamento (como o Máximo Divisor Comum via algoritmo de Euclides e as frações contínuas) possui complexidade polinomial suportável de $\mathcal{O}(n^3)$, onde $n = \log N$ é o número de bits para representar o inteiro N.

O gargalo computacional recai sobre a implementação em nível de portas lógicas do operador unitário de exponenciação modular controlada no hardware quântico. Através de métodos eficientes de multiplicação e adição modular repetida, essa operação inteira pode ser construída utilizando $\mathcal{O}(n^3)$ portas lógicas universais [NC10]. Somada à complexidade da QFT no registrador de medição ($\mathcal{O}(n^2)$), a complexidade geral do Algoritmo de Shor é assintoticamente limitada a $\mathcal{O}(n^3)$. Esta drástica redução de super-polinomial para polinomial é o que fundamenta a futura ameaça quântica à criptografia moderna.

9 Pesquisas Futuras e Algoritmos de Consulta

O presente trabalho consolidou os fundamentos matemáticos e a prática computacional dos principais pilares da informação quântica, abrangendo desde os estados emaranhados multipartidos (Bell, CHSH e GHZ) até os motores algorítmicos universais de busca (Grover) e estimativa de período via transformada de Fourier (QPE e Shor). Como continuidade desta pesquisa e direcionamento para estudos futuros em nível de pós-graduação, destaca-se o aprofundamento no ecossistema mais amplo de *algoritmos de consulta quânticos* (*quantum query algorithms*), bem como na exploração de suas aplicações práticas em criptoanálise e otimização.

Entre as vertentes teóricas mais promissoras para investigações futuras, destacam-se:

1. **Algoritmo de Simon (Periodicidade sob XOR):** Formulado por Daniel Simon em 1994 [Sim97], este algoritmo é historicamente considerado o elo conceitual que inspirou Peter Shor. Dada uma função caixa-preta $f : \{0,1\}^n \to \{0,1\}^n$ com a promessa de que $f(x \oplus s) = f(x)$ para uma string oculta s, o algoritmo quântico identifica o período s com apenas $\mathcal{O}(n)$ consultas, em contraste com a barreira clássica exponencial de $\mathcal{O}(2^{n/2})$ [NC10]. A futura implementação deste protocolo fornecerá uma base didática direta sobre como a amostragem em superposição linear resolve problemas de subgrupos ocultos abelianos.

2. **Contagem Quântica (*Quantum Counting*):** Proposta por Brassard, Høyer e Tapp em 1998 [Bra+02], consiste na união direta de dois algoritmos desenvolvidos nesta iniciação científica: o operador de difusão de Grover e a Estimação de Fase Quântica (QPE). Ao tratar a iteração de Grover como um operador unitário e aplicar a QPE sobre seus autovetores, é possível estimar com alta precisão o número exato M de soluções existentes no espaço de busca em tempo $\mathcal{O}(\sqrt{N/M})$. Essa técnica é essencial para automatizar a busca de Grover quando a quantidade de alvos não é previamente conhecida.

3. **Problema de Elementos Distintos e Caminhadas Quânticas (*Quantum Walks*):** Como extensão natural do algoritmo BHT, o problema de verificar se todos os elementos de uma lista são distintos (identificando colisões arbitrárias) foi solucionado de forma ótima por Andris Ambainis utilizando caminhadas quânticas em grafos de Johnson. O estudo deste formalismo permitirá expandir a análise para além do modelo padrão de circuitos de portas lógicas, adentrando a dinâmica de caminhadas quânticas contínuas e discretas.

4. **Algoritmos Pioneiros de Deutsch-Jozsa e Bernstein-Vazirani:** Representam os primeiros exemplos históricos de oráculos determinísticos onde a computação quântica supera algoritmos clássicos com uma única consulta ($\mathcal{O}(1)$), explorando o paralelismo quântico e o cancelamento exato por interferência de fase.

Para sintetizar o panorama comparativo entre os limites clássicos e as acelerações quânticas dos algoritmos discutidos, a Tabela 2 apresenta o resumo de complexidade de consulta (*Query Complexity*):

Algoritmo	Problema Central	Complexidade Clássica	Complexidade Quântica
Deutsch-Jozsa	Função Constante vs. Balanceada	$\mathcal{O}(2^{n-1}+1)$ (det.)	$\mathcal{O}(1)$ (exato)
Bernstein-Vazirani	Descoberta de String Oculta $a \cdot x$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Simon	Período Oculto sob XOR	$\mathcal{O}(2^{n/2})$	$\mathcal{O}(n)$
Grover	Busca em Dados Não Estruturados	$\mathcal{O}(N)$	$\mathcal{O}(\sqrt{N})$
BHT	Colisão em Função 2-para-1	$\mathcal{O}(N^{1/2})$	$\mathcal{O}(N^{1/3})$
Contagem Quântica	Número de Soluções M	$\mathcal{O}(N)$	$\mathcal{O}(\sqrt{N})$
Shor	Fatoração / Logaritmo Discreto	Super-polinomial	$\mathcal{O}(n^3)$ (polinomial)

Tabela 2: Quadro comparativo da complexidade assintótica de consulta entre modelos clássicos e quânticos.

Além do desenvolvimento analítico e simulação destes protocolos em bibliotecas avançadas (como Qiskit e PennyLane), pesquisas futuras poderão contemplar o estudo de algoritmos quânticos variacionais híbridos para a era NISQ (*Noisy Intermediate-Scale Quantum*), tais como o VQE (*Variational Quantum Eigensolver*) voltado à química quântica e o QAOA (*Quantum Approximate Optimization Algorithm*) aplicado a problemas combinatórios em grafos, consolidando a ponte entre a teoria fundamental e a computação quântica aplicada.

Referências

[Cla+69] John F. Clauser, Michael A. Horne, Abner Shimony, and Richard A. Holt. Proposed experiment to test local hidden-variable theories. *Physical Review Letters*, 23(15):880–884, 1969.

[GHZ89] Daniel M. Greenberger, Michael A. Horne, and Anton Zeilinger. Going beyond bell's theorem. In Menas Kafatos, editor, *Bell's Theorem, Quantum Theory and Conceptions of the Universe*, pages 69–72. Kluwer Academic, 1989.

[Sho94] Peter W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134. IEEE, 1994.

[Kit95] A. Yu. Kitaev. Quantum measurements and the abelian stabilizer problem. *arXiv preprint quant-ph/9511026*, 1995.

[Gro96] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC '96)*, pages 212–219, 1996.

[BHT97] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and one-way functions. In *Automata, Languages and Programming (ICALP '97)*, volume 1256 of *Lecture Notes in Computer Science*, pages 153–165. Springer, 1997.

[Sim97] Daniel R. Simon. On the power of quantum computation. *SIAM Journal on Computing*, 26(5):1474–1483, 1997.

[Bra+02] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp. Quantum amplitude amplification and estimation. In *Quantum Computation and Information*, volume 305 of *Contemporary Mathematics*, pages 53–74. American Mathematical Society, 2002.

[SM07] Smite-Meister. Bloch sphere. `https://commons.wikimedia.org/wiki/File:Bloch_sphere.svg` (2026-01-15).

[NC10] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition, 2010.

[Abh+22] J. Abhijith et al. Quantum algorithm implementations for beginners. *ACM Transactions on Quantum Computing*, 3(4), 2022.

[Qul24] Qulacs Development Team. Quantum circuit diagram. `https://dojo.qulacs.org/en/latest/notebooks/1.4_quantum_circuit_diagram.html` (2026-01-13).

[Gru26] Grupo de Computação Quântica UFSC. Grupo de computação quântica ufsc. `http://gcq.ufsc.br/` (2026-02-03).

[IBM26] IBM Quantum. Qiskit documentation. https://quantum.cloud.ibm.com/docs/ (2026-02-03).

[Lei26a] Felipe Junqueira Leite. Introdução à computação quântica: Fundamentos e algoritmos quânticos (caderno de implementação). https://colab.research.google.com/drive/1kZGYeHcZO14jM8A2tuBPwZPFd99C_N-d?usp=sharing (2026-08-30).

[Lei26b] Felipe Junqueira Leite. Quantum-algorithms. https://github.com/felipejunqueira/Quantum-Algorithms (2026-02-18).